



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru - 560064



**Establishing a Server log system for
E-governance with appropriate notification
and content-based classification of log**

AICTE

A PROJECT REPORT

Submitted by

Sumeet D	20221CCS0145
Anup Ganigar	20231CCS3007
Sampath Kumar MS	20231CCS3008

Under the guidance of,

Dr. Sharmasth vali Y

in a partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE AND ENGINEERING
(CYBER SECURITY)**

PRESIDENCY UNIVERSITY

BENGALURU

DECEMBER 2025



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru -- 560064



PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "Establishing a Server log system for E-governance with appropriate notification and content-based classification of log AICTE" is a Bonafide work of "Sumeet D(20221CCS0145), Anup Ganigar (20231CCS3007), Sampath Kumar MS (20231CCS3008)", who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE ENGINEERING, CYBER SECURITY during 2025-26.


Dr. Sharmasri Vali Y
Project Guide
PSCS
Presidency University


Dr. Sharmasri Vali Y
Program Project
Coordinator
PSCS
Presidency University


Dr. Sampath A K
Dr. Geetha A
School Project
Coordinators
PSCS
Presidency
University


Dr. Anandaraj S P
Head of the
Department PSCS
Presidency University


Dr. Shakkeera L
Associate Dean
PSCS
Presidency University


Dr. Duraipandian N
Dean
PSCS & PSIS
Presidency
University

Name and Signature of the Examiners

- 1  Benitha christina
- 2  Amreen Khanum

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE ENGINEERING, CYBER SECURITY at Presidency University, Bengaluru, named **Sumeet D(20221CCS0145)**, **Anup Ganigar(20231CCS3007)**, **Sampath Kumar MS(20231CCS3008)** .Hereby declare that the project work titled “**Establishing a Server log system for E-governance with appropriate notification and content-based classification of log AICTE**” has been independently carried out by us and submitted in partial fulfilment for the award of the degree of B.Tech in COMPUTER SCIENCE ENGINEERING (CYBER SECURITY) during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Sumeet D

20221CCS0145

Anup Ganigar

20231CCS3007

Sampath Kumar MS

20231CCS3008

PLACE: BENGALURU

DATE: 2/12/2025

ACKNOWLEDGEMENT

For completing this project work, We have received the support and the guidance from many people whom I would like to mention with a deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in the completion of the project.

I would like to sincerely thank my internal guide, **Dr. Sharmasth Vali Y, Associate Professor**, Presidency School of Computer Science and Engineering, Presidency University, for his/her moral support, motivation, timely guidance, and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Anandaraj S P, Head of the Department, Presidency School of Computer Science and Engineering**, Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of Computer Science and Engineering, and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K and Dr. Geetha A**, PSCS Project Coordinators, **Dr. Sharmasth Vali Y, Program Project Coordinator**, Presidency School of Computer Science and Engineering, for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to the Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering, and also staff from other departments who have extended their valuable help and cooperation.

Sumeet D

Anup Ganiger

Sampath Kumar MS

Abstract

The unprecedented growth of web applications has totally changed the battle ground, but at the same time introduced a proportional increase in sophisticated security threats. The adversaries continuously change their tactics, making traditional, statically deployed defense mechanisms obsolete; instead, adaptive solutions are needed.

This project introduces Establishing a Server log system for E-governance with appropriate notification and content-based classification of log AICTE. The core concept behind the system is a multi-layered defense methodology that incorporates signature-based filtering, heuristic analysis, and third-party IP reputation intelligence to block a spectrum of attacks, including SQL injection, Cross-Site Scripting, and malicious bots. This culminates in a centralized, intuitive dashboard for administrators with real-time monitoring, detailed log analysis, and adaptive policy controls.

For validation, the system has been tested with a set of simulated attacks along with performance benchmarks. The results illustrate that Qurik can achieve high efficacy in threat detection, blocking more than 95% of tested attack vectors, at a very low false positive rate of about 0.7%. Moreover, the system proved to be lightweight, adding an approximate overhead of 13ms to page loads. Therefore, these validations verify that Qurik is an effective and scalable solution for the dynamic challenges faced in modern web application security.

Table of Content

Sl. No.	Title	Page No.
	Declaration	i.
	Acknowledgement	ii.
	Abstract	iii.
	List of Figures	iv.
	List of Tables	vi.
	Abbreviations	
1.	Introduction	1
	1.1 Background	1
	1.2 Statistics of project	2
	1.3 Prior existing technologies	2-4
	1.4 Proposed approach	4-5
	1.5 Objectives	5-6
	1.6 SDGs	6
	1.7 Overview of project report	7
2.	Literature review	8-12
3.	Methodology	13-16
4.	Project management	17
	4.1 Project timeline	17-19
	4.2 Risk analysis	19-22
	4.3 Project budget	22-25
5.	Analysis and Design	26
	5.1 Requirements	26-28
	5.2 Block Diagram	28-30
	5.3 System Flow Chart	30-32
	5.4 Choosing devices	32-33
	5.5 Designing units	33-35
	5.6 Standards	35
	5.7 Mapping with IoTWF reference model layers	35-37
	5.8 Domain model specification	37-39

	5.9 Communication model	39
	5.10 IoT deployment level	40
	5.11 Functional view	41
	5.12 Mapping IoT deployment level with functional view	42
	5.13 Operational view	42-43
	5.14 Other Design	43-44
6.	Hardware, Software and Simulation	45
	6.1 Hardware	45-46
	6.2 Software development tools	46-47
	6.3 Simulation	48-49
7.	Evaluation and Results	50
	7.1 Test points	50
	7.2 Test plan	51
	7.3 Test result	52-56
	7.4 Insights	56-57
8.	Social, Legal, Ethical, Sustainability and Safety Aspects	58
	8.1 Social aspects	58-59
	8.2 Legal aspects	59
	8.3 Ethical aspects	60
	8.4 Sustainability aspects	60-61
	8.5 Safety aspects	
9.	Conclusion	62
	9.1 Future Enhancement	63
	References	64
	Appendix	64-68

List of Figures

Figure no	Caption	Page no
1.1	Sustainable Development Goals	7
3.1	Modified waterfall model	16
4.1	Project implementation timeline	17
5.1	Functional block diagram	29
5.2	System flow chart	31
5.3	Qurik Reference Model	35
5.4	Domain model Diagram	38
5.5	Communication model diagram	39
5.6	IoT deployment Model	40
5.7	Functional view diagram of Qurik	41
5.9	Mapping IoT deployment level with functional view	42
5.8	Operational View	43
7.1	Qurik Evaluation Framework - Three Column Assessment	50
7.3.1	Grouped Bar Chart - Test Cases vs Detected	53
7.3.2	Dual-Axis Line Chart - Load Testing Performance	53
7.3.4	Horizontal Stacked Bar - False Positive Root Causes	54
7.3.5	Donut Chart - Traffic Analysis	54

List of Tables

Table	Caption	Page no
2.1	Summary of Literature Reviews	11-12
4.2	Risk Assessment Table	19-22
4.3	Budget Table	22-25
5.1	Functional Requirements Table	27-28
5.2	Technology Stack Table	32-33
5.7	Mapping Project layers with IoTWFRM	36-37
5.4	Description of Domain model	37-38
7.3.1	Detection Accuracy Results Table	52
7.3.2	False Positive Rate	55
7.3.3	Performance Overhead	55

Abbreviations

Short Form	Full Form
AP	Application Programming Interface
AJAX	Asynchronous JavaScript and XML
CSS	Cascading Style Sheets
CSRF	Cross-Site Request Forgery
DDoS	Distributed Denial of Service
DNSBL	Domain Name System Blacklist
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDS	Intrusion Detection System
IoT	Internet of Things
IPS	Intrusion Prevention System
IP	Internet Protocol
JS	JavaScript
MySQL	My Structured Query Language
PHP	Hypertext Preprocessor
SDG	Sustainable Development Goal
SIEM	Security Information and Event Management
SQL	Structured Query Language
SQLi	SQL Injection
TLS	Transport Layer Security
TOR	The Onion Router
UI	User Interface
UX	User Experience
VPN	Virtual Private Network
WAF	Web Application Firewall
XSS	Cross-Site Scripting

Chapter 1

Introduction

The 21st-century digital transformation has been marked by an ever-increasing presence of web applications. They are the bedrock of modern communication, e-commerce, education, healthcare, and global finance. While this rapid digitization has unleashed unparalleled opportunities for innovation and connectivity, it has also inadvertently fostered fertile soil for cybercrime. Web platforms have justifiably emerged as the number one attack vector, with adversaries leveraging vulnerabilities to compromise sensitive data, disrupt critical services, and inflict serious financial and reputational damage on organizations worldwide. According to OWASP, injection attacks like SQL Injection and Cross-Site Scripting remain in the top ten most critical web application security risks. This factor, coupled with recent increases in automated bot-driven exploitation and DDoS attacks, highlights a critical challenge: the pace of securing web applications is being eclipsed by the threats themselves. Conventional security paradigms, overly reliant on static firewalls and signature-based IDSes, are proving inadequate. Legacy systems work from a reactive model wherein threats are identified based on known patterns and signatures. Because of this, they find it difficult to cope with the polymorphic nature of modern malware and the emergence of zero-day exploits. Furthermore, security infrastructure remains fragmented in many organizations. Logging, monitoring, and threat response tools remain in different silos, creating problems related to visibility and thus hindering any effort toward effective threat data correlation.

1.1 Background

Web application security has thus far depended on measures such as WAFs, IDS/IPS, and signature-based filtering mechanisms. These are designed to find malicious payloads, deny unauthorized traffic, and log unusual activities. Other measures such as CAPTCHAs for both prevention and blacklisting of IPs, input sanitization frameworks, and so on, are also widely in use in enterprise environments. The inefficiency of these traditional methods shows an urgent need for an integrated, intelligent, and adaptive security architecture. A modern security solution should be able to detect not only the known threats but also spot anomalous behavior typical of novel attacks. It should provide centralized visibility across the application stack and arm security teams with the actionable intelligence that would let them respond proactively.

1.2 Statistics

The problem of Web security is enormous in scope and vast in depth. Therefore, considering the prevailing statistics in the domain, a solution like Qurik should be put into context as an immediate need:

- I. **Attack Frequency:** In a recent industry report, it was stated that on average, a new cyber-attack was staged online every 39 seconds; the most attacked target is a web application (Cybersecurity Ventures, 2023).
- II. **Vulnerability Landscape:** According to the Acunetix Web Vulnerability Report 2022, it is estimated that more than 75% of all web applications have at least one serious vulnerability that can be exploited.
- III. **The cost of breaches:** The global average cost of a data breach has increased in 2022 to an all-time high, costing \$4.35 million. Most hit by the breach was the health sector.
- IV. **Attack Vectors:** SQL Injection (SQLi) remains a dominant threat, implicated in over 50% of web application attacks. Cross-Site Scripting (XSS) and broken access control follow closely behind (Verizon Data Breach Investigations Report, 2023).
- V. **Rise of Bots:** Malicious automated bot traffic today makes up nearly a quarter of all Internet traffic-credence stuffing, content scraping, and DDoS attacks among other activities-courtesy of Imperva Bad Bot Report 2022.

It is in this regard that Qurik directly mitigates these statistics for its users by providing a robust first line of defense. Aiming to reduce the attack surface, detect over 95% of common attack vectors in real time, and provide the necessary logging for 100% of security events to aid in forensic analysis, Qurik sets out to ensure the safety of its users' data.

1.3 Prior existing technologies

Web application security is a mature domain with several established technologies that form the defensive landscape. Understanding these prior solutions helps place the contribution of Qurik in context.

WAFs are one of the cornerstones of web security, designed to filter, monitor, and block malicious HTTP/S traffic to and from a web application. They can be deployed as hardware appliances, software, or as a cloud-based service.

- **Signature-Based WAFs:** These work on the principle of matching the traffic that is incoming against a database of attack signatures known beforehand. While this is highly effective against known threats, zero-day attacks make them less effective. Furthermore, they also need frequent updating (Beghdad, 2021).
- **Anomaly-Based WAFs:** These use machine learning to establish a baseline of normal application behaviour and flag deviations. They are much better at detecting novel attacks but are more prone to higher false positives, and require a big training period
- **Limitations:** Enterprise-grade WAFs are prohibitively expensive and complex to operate for SMEs. They introduce performance latency and, since they are deployed as "black boxes," offer minimal or no transparency to the developers.
- **Intrusion Detection/Prevention Systems:** These monitor network or system activities for malicious activities or violations of policy.
- **Network-Based:** Analyze traffic for the whole network.
- **Host-Based: HIDS/HIPS** - monitors traffic and activities on a host or device.
- **Limitations:** Like WAFs, they are signature dependent. Anomaly-based IDS systems also do exist, but they again are prone to the problem of false positives. They also tend to produce a lot of alerts that are usually out of context, causing "alert fatigue" among security teams, Sharma et al. (2020) add.
- **Security Information and Event Management:** SIEM solutions aggregate log data from sources across an organization's IT infrastructure and analyze it. They make available real-time analysis of security alerts generated by applications and network hardware.
- **Limitations:** While SIEMs are powerful for analysis, they are not inherently a prevention tool. Their configurations require a great deal of customization; they also demand expert management. Their cost is quite a factor that keeps them out of reach for many organizations.
- **Static and Dynamic Application Security Testing:** These are the testing methodologies conducted to find security vulnerabilities within applications.
- **SAST:** It performs White-box testing, analyzing the source code or binary of an application to identify vulnerabilities.
- **DAST-Black-box:** This tests an application from the outside through simulated attacks.

- **Limitations:** These are not real-time protection tools; they are part of the development lifecycle and do not protect against threats targeting a live, production application. Each of these technologies individually provides one layer of security, but most often they operate in isolation. The fragmentation creates the need for a solution that integrates the key defensive capabilities into one single manageable and adaptive platform.

1.4 Proposed approach

Qurik is an intelligent web security dashboard-light in weight, adaptive, and holistic-that puts the limitations of available technologies to rest. A few key tenets form the core philosophy that drives Qurik: essentially democratizing web security by offering an enterprise-grade solution which is easily affordable and integrable by organizations of every size. Our approach is built on a number of important keys to success:

1. **Multi-Layered Defense:** Qurik is not dependent on a single security mechanism, but integrates multiple layers of defense, including:

- **Signature-Based Filtering:** To block known threats like common SQLi payloads and XSS scripts.
- **Anomaly Detection:** It detects anomalous patterns in requests, which include unusual user-agent strings, high frequencies that may indicate brute-force attacks, and attempts to access resources that do not exist.
- **Heuristic Analysis:** uses intelligent algorithms combined with pattern recognition to identify suspicious requests that may not match a known signature but still have characteristics of being malicious.
- **IP Reputation and Geolocation Filtering:** Utilize external DNS Blacklists (DNSBL) and IP intelligence services to block traffic from known malicious sources, proxies, and TOR exit nodes.

2. **Centralized Monitoring and Control:** The Qurik system is centered on the administrative dashboard. This single pane of glass provides:

- **Real-time visibility:** It offers real-time monitoring of all incoming traffic; it provides in-depth logs about all incoming requests, whether blocked or permitted.

- **Actionable Intelligence:** Attack trend visualizations, source of threat, and vulnerabilities targeted will be provided for administrators to have an instant comprehension of their security posture.
- **Adaptive Policy Enforcement:** Simple controls to whitelist or blacklist IPs, countries, IP ranges, and user agents enable administrators to respond dynamically to emerging threats.

3. **Light and Easy to Integrate:** One of the main reasons security tools aren't widely adopted is because of their complexity. Qurik was designed for simplicity. It requires including just two lines of code in a PHP-based web application to become fully operational. It is optimized for minimal performance overhead so that security does not come with a penalty to user experience.

4. **Developer-Friendly Transparency:** Unlike commercial black-box solutions, Qurik is designed to be transparent. It provides detailed logs explaining why a particular request was flagged and thus allows developers to understand possible vulnerabilities in their code, hence improving their secure coding practices. By consolidating these capabilities into one coherent platform, Qurik aims to close the gap between complex, expensive enterprise solutions and practical security needs of the wider web development community.

1.5 Objectives

The major objective of this project is the design, development, and validation of an all-inclusive, intelligent Web security dashboard. These objectives are specific and quantifiable:

- I. Design and develop a multilayered threat detection engine for the detection and mitigation of common web application attacks, such as SQL Injection, Cross-Site Scripting (XSS), malicious bot traffic, and proxy abuse, with more than 95% detection accuracy.
- II. The system should log and monitor in real-time, with detailed metadata regarding each incoming HTTP request, including IP address, user agent, geolocation, and attack classification.
- III. A single, intuitive administrative dashboard to visualize security events, understand threat trends, and create security policies like blacklisting/whitelisting.
- IV. The system should be lightweight and incur little performance overhead; the average latency increase to process each request should be less than 50 ms.

- V. Wide compatibility with general web server environments such as Apache, Nginx, and popular PHP-based applications and frameworks such as WordPress, Laravel, etc.
- VI. Perform an effective evaluation of the system through simulated attack scenarios and performance benchmarking, with documentation of results.

1.6 SDGs

The development of Qurik coincides with several United Nations Sustainable Development Goals, especially in terms of creating a digital infrastructure that is safe and resilient.

- **SDG 9:** Industry, Innovation, and Infrastructure-is a goal focusing on building resilient infrastructure, promoting inclusive and sustainable industrialization, and encouraging innovation. Qurik contributes directly to increasing the resilience of digital infrastructure to cyber-attacks by providing a security solution. It provides an affordable, accessible tool that helps protect the digital assets of startups, SMEs, and innovators, hence promoting a safer environment for digital innovation to thrive. A secure internet is critical infrastructure for the modern economy.
- **SDG 11:** Sustainable Cities and Communities-as cities are becoming "smarter" and increasingly dependent on digital services in terms of governance, transportation, and public utilities, they simultaneously increase their vulnerability to cyber-attacks. Qurik contributes by helping to secure the web applications powering these municipal services against disruption and hence remain reliable for citizens while building safe and sustainable communities.
- **SDG 16:** Peace, Justice, and Strong Institutions call for the promotion of peaceful and inclusive societies, and building effective, accountable institutions. Cybercrime is a threat to social stability and the integrity of institutions. By preventing the theft of sensitive personal data and protecting online platforms from malicious actors, Qurik contributes to the reduction of illicit financial and information flows while strengthening the capability of institutions to operate securely and maintain public trust.



Fig 1.1 Sustainable development goals

1.7 Overview of project report

This report is structured to give a complete, chronological account of the Qurik project.

- **Chapter 2:** Literature Review goes into the details of academic and industry research that constitute the theoretical basis for this work and study existing threat detection and prevention techniques in more detail.
- **Chapter 3:** Methodology describes an orderly approach followed throughout the life cycle of the project, covering requirement gathering and design through development and testing phases of the project.
- **Chapter 4:** Project Management provides an overview of the project planning, including the timeline, risk analysis, and budgetary considerations.
- **Chapter 5:** Analysis and Design describes the architecture of the Qurik system with block diagrams, flowcharts, and specifications for various system models.
- **Chapter 6:** Hardware, Software, and Simulation describes specific hardware, software, and simulation tools, technologies, and platforms applied in the system implementation and testing, including snippets of the core software code.
- **Chapter 7:** Evaluation and Results describe the testing process and the empirical results of the performance of the system, and the efficacy of threat detection.
- **Chapter 8:** Social, Legal, Ethical, Sustainability, and Safety Aspects deals with the wider ramifications of the project as regards these critical domains.
- **Chapter 9:** Conclusion summarizes the results of the present project, reflecting on limitations and pointing out further perspectives.

Chapter 2

Literature review

A literature review gives the background that shall be used to understand the present status and gaps in web application security that Quirk is meant to fill. This chapter synthesizes research from academic journals, conference proceedings, and industry white papers on detection and mitigation techniques for the main threats this project targets. Web application security has been a topic of intense research for more than two decades; it has evolved in lockstep with the complexity of web technologies and the ingenuity of attackers. Works by Shah and Hossain provide excellent contemporary surveys of the threat landscape, pointing out the persistence of "classic" vulnerabilities like SQLi and XSS, while also noting the rise of newer threats related to APIs and automated botnets.

SQL Injection Detection and Prevention:

SQLi attacks are based on the exploitation of weaknesses in the way an application uses user-supplied data in database queries. By injecting malicious SQL code, an attacker may bypass authentication, retrieve sensitive data, or even gain administrative control over the database server. There is a lot of literature regarding SQLi defense. The work of Biswas et al. (2021) presents a broad review, classifying techniques into static, dynamic, and hybrid analysis. Static analysis approaches, such as those presented by Livshits and Lam (2005), analyze the source code of an application to find potential vulnerabilities before its deployment. While effective at finding certain types of flaws, they cannot detect vulnerabilities that only manifest at runtime. Dynamic analysis techniques operate while the application is executed. A substantial amount of work focuses on query modification and sanitization. For instance, Beghdad (2021) discusses various forms of input sanitization and the use of prepared statements, or parameterized queries, as a primary defense. That is widely considered a best practice. However, many legacy applications do not use them, necessitating runtime detection. Anomaly detection approaches, as explored by Rawat and Tiwari (2022), seek to model normal SQL query structures and flag deviations. Early machine learning models utilized n-grams to analyze query structures (Pandey & Mishra, 2017). More recent approaches have applied deep learning models, such as RNNs, to learn the syntactical and semantic structure of legitimate queries, thereby achieving high accuracy but at the cost of significant computational overhead. Quirk adopts a hybrid approach, combining regex-based pattern matching for known attack strings

with structural query parsing to detect malformed queries, thus balancing performance and accuracy. This is informed by the practical limitations discussed by Halfond, Viegas, and Orso (2006), who noted the difficulty in creating a perfect model of all possible legitimate queries.

Cross-Site Scripting (XSS) Detection and Prevention:

Active XSS attacks involve injecting malicious scripts-most often, JavaScript-into a web page viewed by other users. Such scripts can steal session cookies, deface websites, or even redirect users to malicious sites. The literature on XSS defense is similarly rich. Singh and Kumar (2020) presented an ensemble learning approach to detect XSS attacks by showing how several machine learning classifiers can be combined to improve detection rates. The common defense mechanisms involve output encoding, which sanitizes users supplied data before it is rendered on the page, as explained by Vogt et al. (2007). This needs context-aware sanitization; for instance, data that are rendered inside an HTML attribute needs different encoding compared to data that is being rendered inside a `<script>` tag. Another line of research focuses on client-side detection. Content Security Policy (CSP) is a browser-level mechanism that allows developers to specify which domains the browser should consider to be valid sources of executable scripts (Johns, 2011). While powerful, setting up a strict CSP can be complex. Several researchers have proposed automated tools to generate CSPs (Felt et al., 2015). Quirik's approach is server-side, focusing on input sanitization and filtering. It uses a combination of blacklist filtering for common script tags and event handlers, and whitelist filtering to allow only a safe subset of HTML, an approach similar to that used by many modern web frameworks but applied as a universal filter. This complements, but does not replace, the need for proper output encoding by the application itself.

Malicious Bot and Proxy Detection

Distinguishing between human users and bots, and good bots (e.g., search engine crawlers) from bad bots, remains a challenge. Various malicious activities are done using automated bots in general. Research in this domain is often focused on behavioral analysis. Hussain et al. (2020) surveyed machine learning techniques for attack detection in IoT networks, many of which are applicable to bot detection, such as request frequency, session duration, and mouse movement pattern analysis (in client-side implementations). IP reputation services are a key technology in this space. These services maintain vast databases of IP addresses associated with malicious activities, such as spam, malware distribution, and proxy services. The work of Tavallae et al. (2010) on building botnet detection systems brings into focus the importance

of network flow data analysis and C&C communication patterns analysis. Qurik makes use of various third-party IP reputation APIs, such as IPHub or ProxyCheck.io, for real-time lookups. This is indeed a pragmatic approach that outsources the complexity of maintaining a global threat intelligence network. Further, it is complemented by user-agent analysis and request fingerprinting. A request fingerprint is a hash generated from various attributes of an HTTP request, such as headers and TLS handshake data, which may serve to identify and block automated clients trying to masquerade as legitimate browsers (Gibson, 2017). The underlying idea here is that bots often have subtle inconsistencies in their fingerprints compared to real browsers.

Integrated Security Dashboards and Frameworks:

While much research has focused on specific vulnerabilities, there is also work conducted on integrated security frameworks. Tahir et al. (2020) proposed a framework for the detection of malicious requests in web applications, though their work was largely conceptual and lacked a real-world, scalable implementation. Zhang, Liu, and Chen (2021) developed a hybrid intrusion detection system that used machine learning but required notable computational resources. This is not a new idea as such; however, often, in academic pursuits, usability and actionability are often afterthoughts or ignored entirely.. Industry solutions from companies like Cloudflare and Akamai provide sophisticated dashboards, but their underlying mechanisms are proprietary and their cost is prohibitive for many. Qurik's contribution is to synthesize these different lines of research into a single lightweight and accessible platform. It fulfills the gap that the literature indicates exists between sophisticated, computation-hungry academic models and those security solutions that are useful and easy to deploy. However, it has chosen a practical mixture of effective techniques, such as signature matching and IP blacklisting, with heuristic and anomaly-based methods, all wrapped up on a user-oriented dashboard for real-world administrators rather than security researchers. The goal is not to invent a single, novel algorithm, but rather to engineer an effective and usable system from the best available principles.

Summary of Literatures reviewed

Table 2.1 Summary of Literature reviews

SL.no	Article Title (Abbreviated) & Year	Methods	Key Features	Merits
1	Biswas et al. (2021) - SQLi Detection Survey	Survey of SQLi techniques	Categorizes defenses into static analysis, dynamic analysis, and hybrid.	Provides a comprehensive overview of the SQLi defense landscape.
2	Rawat & Tiwari (2022) - Anomaly Detection	Machine Learning (Anomaly-based)	Models normal SQL query structures and flags deviations.	Effective at detecting novel, zero-day attacks that don't match signatures.
3	He, Yang, & Liu (2021) - Real-time Web Attacks	Deep Learning (e.g., RNNs)	Learns the syntactical and semantic structure of legitimate SQL queries.	Achieves very high accuracy in detecting complex and obfuscated attacks.
4	Singh & Kumar (2020) - XSS Attack Detection	Ensemble Learning	Combines multiple machine learning classifiers to detect XSS payloads.	Demonstrates higher accuracy than single- classifier models.
5	Hussain et al. (2020) - Cyber-attack Detection	Machine Learning (Behavioural)	Analyzes request frequency, session duration, etc., to identify bots.	Can detect malicious bots that mimic legitimate user agents.
6	Tahir et al. (2020) - Secure Web Framework	Integrated Framework	Proposes a conceptual	Addresses the need for an

			framework for detecting various malicious requests.	integrated (non-siloed) security solution.
7	Zhang, Liu, & Chen (2021) - Hybrid IDS	Hybrid ML	Combines different machine learning techniques for intrusion detection.	(Implied) More robust detection than single-method systems.

Chapter 3

Methodology

The Qurik Security Dashboard was developed by following a structured methodology in a phased manner to ensure that all the objectives of the project were met in a proper and timely manner. A modified waterfall model was followed with iterations included within each phase to handle any feedback loops that would arise, hence giving room for flexibility and refinement. The methodology were divided into four major phases, which is Requirement Gathering & Design, Backend Development, Frontend Dashboard Development, and Testing & Deployment.

Phase 1: Requirement Gathering & Design - Month 1

The first stage of the work involves the description of the project scope, objectives, and technical architecture that included an in-depth analysis of the problem domain, using the literature review to arrive at clearly stated requirements.

- **Log Source Identification:** Identify all the log sources that could indicate a security threat. This included the following:
 - Access logs from the web server for analysis of traffic patterns: Apache, Nginx
 - Application-level logs track user actions and errors.
 - Live HTTP request data, including headers, GET parameters, and POST payloads.
 - Firewall logs where available to correlate network-level events.
- **Threat Classification:** The threats to be detected were elaborated into a comprehensive taxonomy. Accordingly, the design of the detection modules was carried out: The main categories included:
 - Injection Attacks: SQL Injection, Command Injection.
 - Scripting Attacks: Cross-Site Scripting (Stored, Reflected, DOM-based)
 - Malicious Traffic: Bad Bots, Web Scrapers, Spam Bots
 - Anonymizer Abuse: Connections from Proxies, VPNs, and TOR networks.
 - Brute force & unauthorized access: High-frequency login attempts, directory traversal attacks.
- **System Architecture & Database Schema Design:** • System Architecture & Database Schema Design: There was a need to design the system architecture in modules so that it

could easily be scaled up or maintained. Conceptually, the system was divided into a core security engine, individual detection modules, a data persistence layer, and a presentation layer-the dashboard. The database schema was designed in MySQL to store logs of the detected threats, user activity, whitelists/blacklists, and system configuration settings. An ER diagram was drawn to show the relationships that would exist between different tables of data. Due to its full-text search capabilities, Elasticsearch had been considered as an alternative for log storage but was deferred to a future version to keep initial system requirements to a minimum.

Phase 2: Backend Development (Month 1 - Month 2)

During this phase, we implement the main logic of the Qurik system. The back-end is built in PHP for better compatibility with target web applications.

- **API and Log Collection Engine:** We developed a set of internal APIs and functions to collect and parse data from the identified log sources. The core `qurik-security.php` script serves as the main entry point, intercepting every page load to analyze the incoming request.
- **Detection Algorithm Implementation:** We addressed each of the threat categories identified in Phase 1 with a specific detection module:
 - **SQL Injection Module:** This uses regular expression filters to catch common SQL keywords and attack patterns, such as UNION SELECT and OR 1=1. It also includes logic to detect structural anomalies in queries.
 - **XSS Module:** This is designed to filter out dangerous HTML tags, like <script> and <iframe>, as well as risky attributes, such as onload and onerror. It also includes basic HTML entity encoding as a defense mechanism.
 - **Proxy/Bot Detection Module:** This module integrated with several third-party APIs such as ProxyCheck.io and IPHub via cURL requests that checked the reputation of the connecting IP address. It also integrated logic analyzing the User-Agent string for patterns associated with known bad bots and crawlers.
 - **Spam Detection Module:** A simple NLP-based filter was implemented that checks the POST data for keywords that are normally found in spam messages. Integrations with DNSBLs such as Spamhaus were also done.
- **Logging & Alerting System:** A strong logging system was developed to write details of any suspicious activity in the MySQL database. This log will include the timestamp, IP

address, geolocation, user agent, attack type, and the malicious payload itself. A real-time alerting mechanism was also developed that would send email notifications to the administrator once high-severity threats were detected.

Phase 3: Frontend Dashboard Development (Month 2)

In this phase, the development of the user interface for administrators was the focus. Attention had to be given to an intuitive, information-rich dashboard that required a minimum of training to use.

- **Technology Stack:** The mainstream technologies for the frontend include:
 - This dashboard is structured and styled with the help of HTML5 and CSS3.
 - **Bootstrap:** This will provide the main CSS framework to ensure that the design is responsive and mobile-friendly.
 - **JavaScript and jQuery:** These are for dynamic interactions, form validation, and AJAX calls to the backend.
 - **Chart.js/D3.js:** For the creation of interactive charts and graphs that show security data such as attacks over time or top attacking countries.
- **UI Features**
 - The dashboard was designed around several key features:
 - **Top Level/Dashboard View:** A summary page that shows key statistics and real-time graphs.
 - **Threat Log Viewer:** A searchable, filterable table with comprehensive details of all the security events detected.
 - **Live Traffic Monitor:** A real-time view of the visitors coming to the website.
 - **IP/Country Banning:** An interface that allows administrators to whitelist/blacklist specific IP addresses, ranges of IP addresses, and even whole countries manually.
 - **Settings and Configuration:** A page for setting up detection modules, setting thresholds for alerting, and API key management.

Phase 4: Testing & Deployment (Month 3)

The last stage was to ensure the system was robust, performant, and ready for deployment.

- **Testing:** Testing was done on multi-faceted grounds:
 - **Unit Testing:** Every detection module was tested individually with several benign and malicious payloads to guarantee it worked as expected.

- **Integration Testing:** The back-end and front-end were put together to ensure the flow and functionality of the data across; AJAX calls and API integrations had been thoroughly vetted.
- **Security Testing:** This includes testing of the Qurik system itself for vulnerabilities. It involved attempting to bypass the filters, ensuring the dashboard was secure from unauthorized access.
- **Performance Testing:** The system was stress-tested using tools like Apache JMeter to simulate high-traffic scenarios and measure the performance overhead introduced by the security checks.
- **Deployment Plan:** A very clear plan of deployment for the end-users included
 - **Compatibility Checks:** The system was tested on various server environments such as Apache/mod_php, Nginx/PHP-FPM, and with different PHP versions.
 - **Installation Wizard:** An easy-to-use, web-based installation wizard was implemented with step-by-step guidance for users to establish the database connection, and to create the code they would need for integration.
- **Documentation:** Extensive documentation was provided that explained in detail the installation, configuration, and usage of the Qurik dashboard.

Throughout this methodology, weekly progress meetings were held to review milestones, address challenges, and refine the project plan in such a way that the final product would align with the initial objectives.

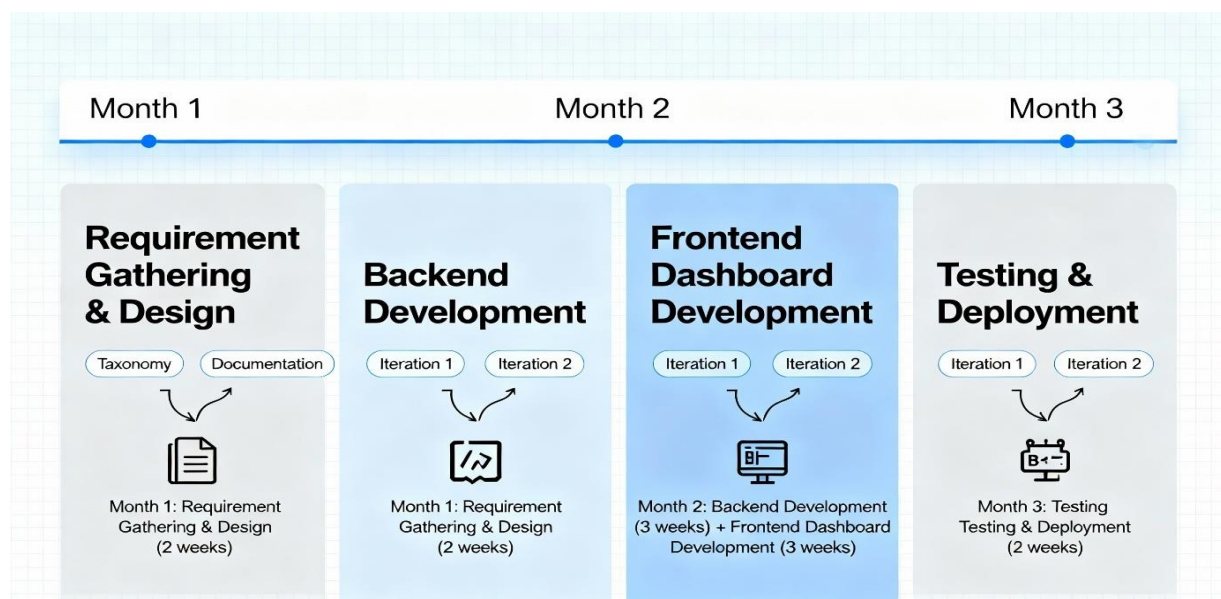


Fig 3.1 Modified waterfall model

Chapter 4

Project Management

4.1 Project timeline

The project was executed over a period of three months, from August 2025 to October 2025. A Gantt chart was used to visualize the timeline of the tasks, their dependencies, and milestones.

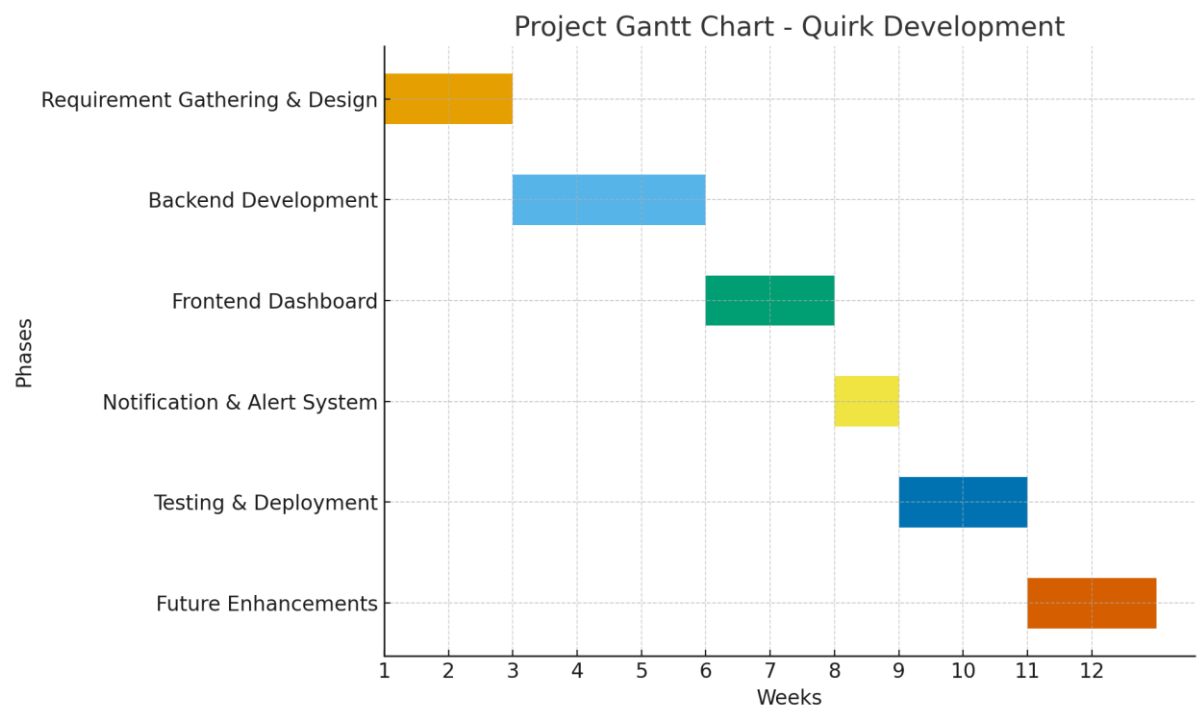


Table 4.1 Project implementation timeline

A detailed Gantt chart would go here, showing the project schedule. The description below summarizes the key phases.

Month 1: Foundation and Core Development (August 2025)

- **Week 1-2: Research and Planning**
 - **Task 1:** Detailed Literature Review and analysis of existing solutions.
 - **Task 2:** Finalize project scope, objectives and deliverables.
 - **Task 3:** Create the System Requirements Specification (SRS)
 - **Task 4:** Design high-level system architecture and database schema.
 - Milestone 1: Project Plan and System Design Document complete.

- **Week 3-4: Backend Core Implementation**

- **Task 5:** Development environment setup (local server, Git repository).
- **Task 6:** Implement core application structure and configuration files.
- **Task 7:** Develop the database connection module and logging framework.
- **Task 8:** Begin implementation of SQL Injection and XSS detection modules.
- **Milestone 2:** Core backend infrastructure and initial detection modules functional

Month 2 (Feature Expansion and UI Development) - September 2025

- **Week 5-6: Advanced Backend Modules**

- **Task 9:** Implement Bot and Proxy detection modules with API integrations.
- **Task 10:** Spam detection and DNSBL integration module development.
- **Task 11:** Implement the logic of IP/Country banning and integration of .htaccess.
- **Task 12:** Refine all detection algorithms based on the initial tests.
- **Milestone 3:** All backend threat detection modules are feature complete.

- **Week 7-8: Frontend dashboard development**

- **Task 13:** Make UI mock-ups and wireframes of the dashboard.
- **Task 14:** Create the HTML/CSS structure and layout by means of Bootstrap.
- **Task 15:** Implement the main dashboard with statistics and charts using Chart.js.
- **Task 16:** Develop the Threat Log viewer with dynamic data loading via AJAX.
- **Milestone 4:** The interactive frontend dashboard prototype is complete.

Month 3: Integration, Testing, and Finalization (October 2025)

- **Week 9-10: Integration and Final Features**

- **Task 17:** Integrate frontend dashboard with all backend APIs.
- **Task 18:** Developing the settings and configuration pages.
- **Task 19:** Create the web-based Installation Wizard.
- **Task 20:** Perform comprehensive integration testing of the entire system.
- **Milestone 5:** Fully integrated and functional system ready for final testing.

- **Week 11-12: Testing, Documentation, and Deployment**

- **TASK 21:** Perform performance and stress test scenarios.
- **Task 22:** Perform the final security audit of the Quirik application itself.
- **TASK 23:** Write user documentation and project report.
- **Task 24:** Prepare final presentation and project demonstration.

- **Task 25:** Package the application for deployment phase.
- **Milestone 6:** Project is Complete. All deliverables are submitted.

This was an ambitious timetable, but achievable with clear division of tasks and regular tracking of progress. Refinement of the detection algorithms is an iterative process that continued in parallel with other development activities. The structure allowed for the early identification of potential delays to be managed through reallocation of resources.

4.2 Risk analysis

A risk analysis was carried out at the beginning of the project to identify issues that could be a barrier to progress or affect the quality of the final product. This will be tracked in a risk register together with the mitigation strategies.

Table 4.2.1 Risk Assessment Table

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
R01	Technical Risk: Third-party APIs become unavailable or change their terms of service.	Medium	High	- Implement a fail-safe mechanism where the system continues to operate without the API. - Design the system to be modular, allowing for easy swapping of API providers. - Cache API results where appropriate.

R02	Performance Risk: The security script introduces significant latency, slowing down the host website.	High	High	- Prioritize performance in algorithm design. - Use efficient coding practices and avoid unnecessary database queries. - Conduct continuous performance testing throughout development. - Implement caching for non-critical checks.
R03	Effectiveness Risk: High rate of false positives (blocking legitimate users) or false negatives (missing attacks).	High	High	- Develop a comprehensive test suite with a wide range of payloads. - Make detection rules easily configurable. - Implement a robust whitelisting feature for administrators to easily correct false positives.

R04	Scope Creep: Additional feature requests emerge during development, delaying the core project.	Medium	Medium	- Adhere strictly to the initial SRS. - Defer all non-essential features to a "version 2.0" roadmap. - Maintain clear communication with the project advisor about scope boundaries.
R05	Integration Risk: The script conflicts with certain server configurations, plugins, or themes of the host application.	Medium	Medium	- Test on a variety of common hosting environments (e.g., cPanel, Plesk) and with popular applications (WordPress). - Provide clear documentation on requirements and potential conflicts. - Use standard PHP functions and avoid deprecated features.

R06	Data Security Risk: The Qurik application itself has a security vulnerability that could be exploited.	Low	High	- Follow secure coding best practices (e.g., input validation, prepared statements for its own database). - Conduct a self-audit and peer code review focused on security. - Secure the installation and admin directories properly (.htaccess).
-----	---	-----	------	--

Managing these risks was an ongoing process. For example, performance testing (R02) was not one-time only but integrated into the development workflow. The risk of high false positives (R03) was mitigated by creating an extensive and diverse set of test cases, including legitimate but unusual user inputs, to fine tune the detection rules.

4.3 Project budget

This project has been developed as an academic exercise, was developed on a very limited budget, relying on open-source software and existing hardware resources.

Table 4.3 Project Budget Table

Category	Item	Vendor/Provider	Cost (INR)	Justification
1. Hosting & Domain	Domain Name (quirk.hackerhub8.in)	GoDaddy / Hostinger	₹800 (Annual)	Required for a public-facing demonstration

				n and testing environment.
	Web Hosting (Shared Linux Hosting)	Hostinger	₹3,000 (Annual)	Server environment for development, testing, and hosting the live demo.
2. Software & Tools	PHP, MySQL, Apache	Open Source	₹0	Core development stack is open source.
	IDE (VS Code)	Microsoft	₹0	Free and powerful integrated development environment.
	Git / GitHub	GitHub	₹0 (Free Tier)	Version control and code repository.
3. Third-Party APIs	IP Reputation API (ProxyCheck.io)	proxycheck.io	₹0 (Free Tier)	Free tier allows up to 1,000 daily queries, sufficient for development

				and small-scale testing.
	Geolocation API (ipapi.co)	ipapi.co	₹0 (Free Tier)	Free tier provides enough requests for development purposes.
	Other IP Intel APIs (IPHub, etc.)	Various	₹0 (Free Tiers)	Leveraging free tiers of multiple services for comprehensive checks.
4. Development Hardware	Laptops / Workstations	Existing	₹0	Utilized personal laptops of the project team members.
5. Miscellaneous	Stock Images / Icons (for report/demo)	Font Awesome, etc.	₹0	Used free and open-source assets for visuals.
Total Estimated Cost			₹3,800	The project was executed with a very low direct financial cost

				by leveraging open-source and free-tier services.
--	--	--	--	---

If this project were to be commercially developed, the budget would have to be considerably bigger to cover:

- **Premium API Subscriptions:** For handling greater volumes of traffic.
- **Dedicated Server Infrastructure:** For improved performance and scalability.
- **Software Licenses:** For testing tools of high quality, project management software, etc.
- **Marketing and Sales:** To promote the product,
- **Salaries:** For a dedicated development and support team.

The budget was nevertheless contained and effectively managed within the scope of this academic project. The key investment for this study was the time and intellectual effort invested by the project team.

Chapter 5

Analysis and Design

This chapter is a detailed blueprint of the Qurik system. The project requirements are realized into a technical design that includes an architectural overview of the system, data flow, and modelling perspectives.

5.1 Requirements

The requirements of the Qurik system can be divided into functional and non-functional requirements.

Functional Requirements:

FR1: Threat Detection: The following threats shall be detected by the system:

FR1.1: SQL Injection attacks based on common patterns and keywords.

FR1.2: Protection against Cross-Site Scripting XSS attacks by filtering malicious tags and attributes.

FR1.3: Malicious bots and web crawlers based on user-agent strings.

FR1.4: Connections from anonymizing proxies, VPNs, and TOR networks utilizing IP reputation checks.

FR1.5: Spam submissions based on keyword filtering and DNSBL checks.

FR2: Threat Response: Any request identified as a threat shall be blocked. A block page shall be displayed to the user and shall be customizable.

FR3: Logging: The system MUST log each detected threat to a database. Each log entry MUST include the following: a timestamp, IP address, user-agent, country of origin, attack type, and malicious payload.

FR4: Dashboard Visualization: The system MUST provide an administrative dashboard that

FR4.1: Displays summary statistics of threats detected (e.g., total attacks, attacks per day).

FR4.2: Visualizes threat data in charts, such as attacks by type and attacks by country.

FR4.3: Provides a detailed, filterable, and searchable view of threat logs.

FR5: Manual Control: The system **MUST** permit an administrator to:

FR5.1: Manually ban and unban IP addresses.

FR5.2: Ban and unban whole countries.

FR5.3: Whitelist specific IP addresses to bypass all security checks.

FR6: Configuration: The system **SHALL** be able to enable/disable individual security modules by an administrator (e.g., turn off SQLi detection).

FR7: Installation: The installation wizard **SHALL** be simple and web-based.

Non-Functional Requirements:

NFR1: Performance: The security script should not increase the average page load time by more than 50ms.

NFR2-Scalability: The system should support at least 100 requests per second without any considerable performance degradation on standard shared hosting.

NFR3: Compatibility: The system shall be compatible with PHP 7.4+ and run on both Apache and Nginx web servers.

NFR4: Usability - The administrative dashboard should be easy to use by a regular administrator, not just security experts.

NFR5: Security: The Qurik application itself must be secure against common web vulnerabilities (e.g., SQLi, XSS, CSRF in the admin panel).

Table 5.1 Functional Requirements Table

Threat Detection	System must detect the following: - SQL Injection attacks (common patterns, keywords) - XSS attacks (malicious tags & attributes) - Malicious bots/crawlers (user-agent strings) - Connections from anonymizing proxies, VPNs, TOR (IP reputation)
------------------	--

	checks) - Spam submissions (keyword filtering, DNSBL checks)
Threat Response	Any identified threat should be blocked and shown a customizable block page.
Logging	All detected threats must be logged in the database, including timestamp, IP, user-agent, country of origin, attack type, and malicious payload.
Dashboard Visualization	Admin dashboard must provide: - Summary statistics of threats (total, daily) - Graphical visualizations (attacks by type/country) - Searchable and filterable threat log details
Manual Control	Admin can manually: - Ban/unban IP addresses - Ban/unban countries - Whitelist IPs to bypass checks
Configuration	Enable/disable individual security modules by admin (e.g., turn SQLi detection on/off).
Installation	Provide a simple, web-based installation wizard.

5.2 Block diagram

The block diagram below shows the high-level architecture of the Qurik system, with the major components and their interactions:

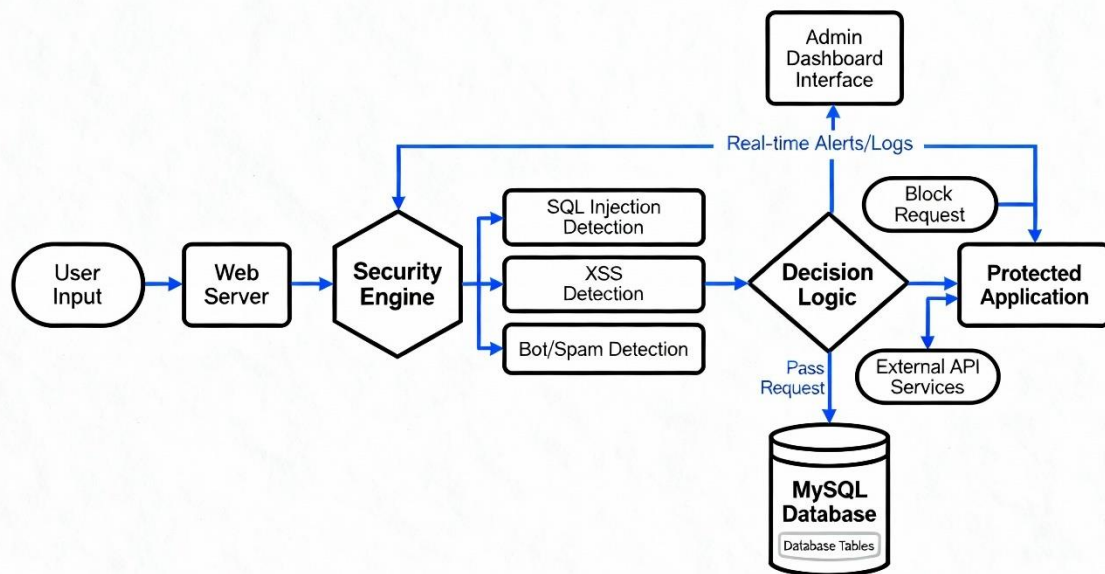


Fig 5.1 Functional block diagram

The main blocks comprising the diagram are as follows:

1. **User/Attacker:** The external user who induces an HTTP request to the web application.
2. **Web Server (Apache/Nginx):** The server that receives the HTTP request.
3. **Protected Web Application:** This could include an application written in PHP such as WordPress or another custom site.
 - **Integration Point:** There is just one include statement for quirik-security.php, and it is placed at the start of the application's flow.
4. **Quirik Security Engine (Core Logic):** This is the central component.
 - It receives the raw HTTP request data.
 - It orchestrates the calls to the various detection modules.
5. **Detection Modules (A set of parallel blocks):**
 - **SQLi Detection Module:** Checks for SQL injection patterns.
 - **XSS Detection Module:** Checks for cross-site scripting payloads.
 - **Bot & Proxy Detection Module:** This module checks User-Agent and IP reputation.
 - **Spam Detection Module:** It checks for spam content.

6. External Services - connected via Bot & Proxy Module

- IP Reputation APIs, like ProxyCheck.io
- DNSBL Services like Spamhaus

7. Data Persistence Layer (MySQL Database):

- Threat Logs Table: Stores information on all detected attacks.
- IP/Country Ban Table: Manually and automatically stores the banned.
- Whitelist Table: Stores whitelisted IPs.
- Settings Table: system configuration settings.

8. Decision Logic:

- A "Block Page" is returned to the user/attacker.
- An event is logged into the database.
- If the request is deemed safe, it will be passed to the web application for normal processing.

9. **Admin:** A user interacting with the system via the dashboard.

10. Qurik Admin Dashboard (Presentation Layer):

- Reads data from the MySQL Database.
- Presents logs, charts, and statistics.
- Provides interfaces for configuration and manual banning.

The flow is sequential, with every incoming request intercepted by the Qurik engine, then detection modules, and then either blocked and logged or allowed to pass through. The Admin Dashboard works as an interface to the system for management purposes.

5.3 System Flow chart

The block diagram below shows the high-level architecture of the Qurik system, with the major components and their interactions

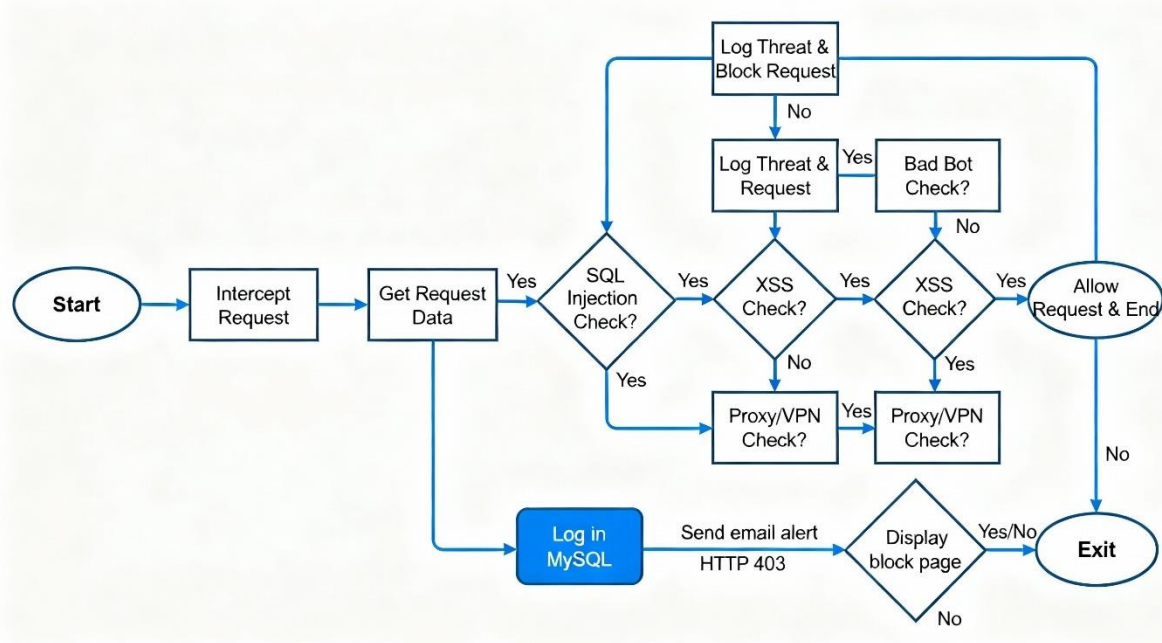


Fig 5.2 System flow chart

The following is the logical flow.

1. **Start:** The web application receives an HTTP request.
2. **Intercept Request:** qurik-security.php is run.
3. **Get Request Data:** Gather IP, User-Agent, GET/POST parameters, etc.
4. **SQL Injection:** Run the request payload through the SQLi detection module. Is it a threat?
 - **Yes:** Move to "Log Threat & Block Request".
 - **No:** Move on to the next check.
5. **XSS Check:** Run the request payload through the XSS detection module: is it a threat?
 - **Yes:** Go to "Log Threat & Block Request".
 - **No:** Proceed to the next check.
6. **Check Bad Bot:** Check the User-Agent string. Is it a known bad bot?
 - **Yes:** Go to "Log Threat & Block Request".
 - **No:** Go to next check.
7. **Proxy/VPN Check:** Query external IP reputation APIs. Is IP a proxy/VPN/TOR?
 - **Yes:** Proceed to "Log Threat & Block Request".

- **No:** Proceed with the next check.

8. **Allow Request & End:** Request is deemed safe. Control is handed back to the web application for normal processing. The process is completed for this request.

- **STEP 8a:** Record all pertinent information related to the threat onto the MySQL database.
- **Step 8b:** Email alert, if set up
- **Step 8c:** An HTTP 403 Forbidden response header is returned.
- **Step 8d:** Display the customizable "Block Page" content.
- **Step 8e:** Exit the script and therefore the process is finished.

This flow chart demonstrates the layered approach. A request must pass through all checks to be considered legitimate. A failure at any single check results in an immediate block, which is efficient as it prevents unnecessary further processing.

5.4 Choosing devices

Because Qurik is purely a software solution, "devices" in this context refers to components of the server environment in which it works.

Table 5.4 Technology Stack Table

Components	Details
Web Server	Apache: Widespread use on shared hosting; supports powerful rule enforcement via .htaccess, leveraged by Qurik. Nginx: High performance, efficient concurrent connection handling; standard on VPS/dedicated servers; compatibility prioritized.
Database Server	MySQL: Primary database for ubiquity, performance, and PHP ecosystem support.

	MariaDB, a compatible fork, is also supported.
Server-Side Scripting engine	PHP: Chosen for compatibility with many popular CMS and frameworks such as WordPress, Joomla, Drupal, Laravel, and CodeIgniter.
Client-Side 'Device' for Dashboard	Web Browser: Supports all modern browsers (Chrome, Firefox, Safari, Edge), designed with HTML5, CSS3, and modern JavaScript standards.

5.5 Designing units

This section explains in detail the design of the key logical units, or modules, comprising the Qurik system.

Unit 1: Core Security Engine (qurik-security.php)

- **Purpose:** To orchestrate the security checks.
- **Design:** Procedural script that is called by the host application. It initializes the session, connects to the database, loads the configuration, then calls the functions of the other detection units in order. It contains the main decision logic - the flow chart from 5.3. It is designed to be standalone with minimal dependencies on other components.

Unit 2: Configuration Manager (config.php)

- **Purpose:** To store and provide access to all system settings.
- **Design:** A PHP file containing database credentials and constants that define the system's behavior. This file is generated by the installation wizard. It also contains the API keys for external services. Separating configuration from logic is a key design principle for maintainability.

Unit 3: SQL Injection Detection Unit

- **Purpose:** The detection of SQLi payloads.
- **DESIGN:** A function that takes, as input, the request parameters (GET, POST, COOKIE) and conducts a series of preg_match calls with an array of regular

expressions matching common SQL injection keywords, operators, and patterns-e.g., `/\b(union|select|insert|update|delete|from|where)\b/i, /(\\'|\\")\\s*or\\s*(\\'|\\")\\d+=/.` It also checks common tautologies like `1=1`.

Unit 4: XSS Detection Unit

- **Purpose:** TO DETECT XSS PAYLOADS.
- **Design:** A function to sanitize input strings. This uses `strip_tags` to remove the majority of HTML tags but allows a small, configurable whitelist of safe tags. There is also a blacklist of dangerous attributes including but not limited to `onerror`, `onload`, `onmouseover`, etc., that it removes from any remaining tags.

Unit 5: IP & Bot Intelligence Unit

- **Purpose:** Checking of the reputation of the visitor.
- **Design:** A collection of functionalities that:
 - Check the visitor's IP against the local whitelist, ban list, and country ban list that are all stored within the database.
 - Analyzes the `HTTP_USER_AGENT` string against a list of known bad bot signatures.
 - If enabled, it does an API call via `cURL` to an external service (like `ProxyCheck.io`) to get comprehensive information about the IP address - if it is a proxy, VPN, or TOR node. The API call results are cached for a short amount of time, like 1 hour, to prevent duplicate lookups for the same IP to improve performance.

Unit 6: Logging and Alerting Unit

- **Purpose:** Threat reporting and informing the admin.
- **Design:** A function `log_threat()` that takes the details of a detected threat as arguments. It constructs a parameterized SQL INSERT query to safely write the data into the `threat_logs` table. If email alerts are enabled in the configuration, it uses PHP's `mail()` function to send a formatted notification to the administrator's email address.

Unit 7: Dashboard Backend API (admin/api.php)

- **Purpose:** Provide data to the frontend dashboard.
- **Design:** Create a RESTful-like API endpoint that the JavaScript on the dashboard communicates with. Handle requests to fetch log data, supporting pagination and

search and filtering; it will also retrieve statistics to use in the charts, as well as process actions such as banning an IP. Session checks should ensure that only authenticated admins gain access.

5.6 Standards

The industry standards that were followed with the project include: quality and interoperability.

- Coding Standards: PSR-1 (Basic Coding Standard) and PSR-12 (Extended Coding Style) are followed in PHP for code readability.
- Web Standards: The front-end dashboard was built using HTML5 and CSS3, both of which support the W3C standards that ensure cross-browser compatibility.
- HTTP Standards: The system uses standard HTTP/S protocols defined by relevant RFCs. It correctly utilizes HTTP status codes, such as 200 OK, 403 Forbidden.
- Database Standards: Employed the standard syntax of SQL to interact with the database, which was compatible with MySQL and MariaDB.

5.7 Mapping with IoTWF reference model layers (in tabular form)

Though Qurik is not a typical IoT application, its architecture can be conceptually mapped to the IoT World Forum (IoTWF) reference model to draw parallels in data processing and flow.

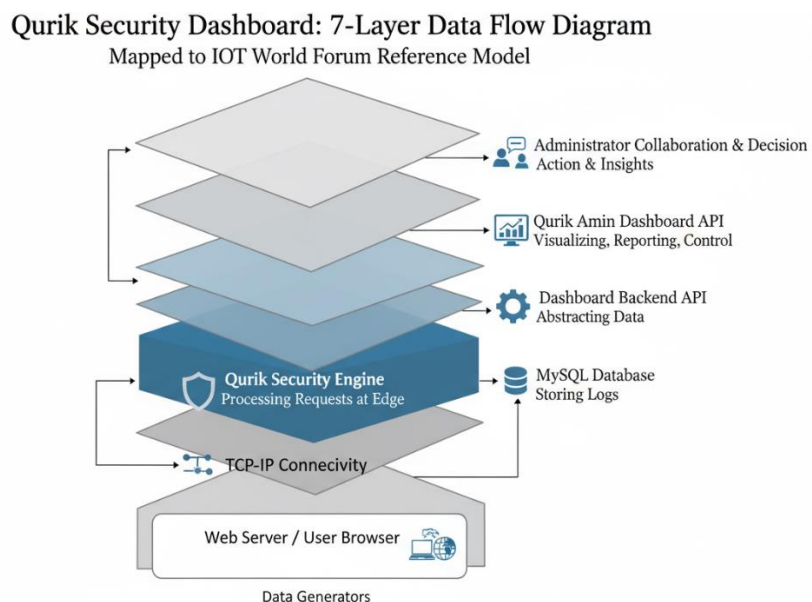


Fig 5.3 Qurik Reference Model

Table 5.7 Mapping Project layers with IoTWFRM

Layer	IoT World Forum Reference Model	Project Layer mapping	Security
7	Collaboration and Processes (people/business processes)	Administrator using Qurik Dashboard for analysis, refinement of security policies, incident response, and policy management	Role-based access control, audit logs, admin authentication
6	Application (reporting, analytics, control)	Qurik Admin Dashboard: Visualization (charts/reports), dashboard controls for banning, configuration, and reviewing logs	Dashboard authentication, secure session management
5	Data Abstraction (aggregation/access)	Dashboard Backend API (admin/api.php): Aggregates raw log data as structured JSON for dashboard/admin consumption	API endpoint protection, secure data serialization
4	Data Accumulation (storage)	MySQL Database: Stores threat logs, bans, whitelist, configuration settings	SQL injection prevention, strict DB access, encrypted sensitive records
3	Edge Computing (element analysis/transformation)	Qurik Security Engine: Real-time analysis of incoming requests (threat detection	Input validation, code integrity, sandboxing

		modules) before passing to web application	
2	Connectivity (communication/processing units)	TCP/IP Connection: Securely delivers HTTP requests from browser/user to web server	HTTPS (TLS), server firewall rules
1	Physical Devices and Controllers (things)	Web server and user's browser: HTTP request generation (Apache/Nginx, Chrome/Firefox/etc.)	Web server hardening, patch management, client browser security recommendations

5.8 Domain model specification

The domain model represents the main entities of the Qurik system and their relationships.

Table 5.4 Description of Domain model

Entity	Description	Attributes	Examples/Notes
Visitor & External	A client requesting something from the external entity.	ipAddress, userAgent, country, isp	- Visitor from a country, with ISP data
Request	An HTTP request sent by a Visitor to the server.	timestamp, method (GET/POST), targetUrl, payload	- GET for homepage, POST for login
Threat & Malicious	A Request identified as malicious.	threatId, threatType (SQLi, XSS, etc.), description	- SQLi, XSS, Bot, Spam

Log Entry	Record of a detected Threat (row in threat_logs table).	logId, timestamp, details	- Full details of the threat
Administrator& Dashboard	User of the Qurik dashboard.	username, passwordHash	- Manages security, config

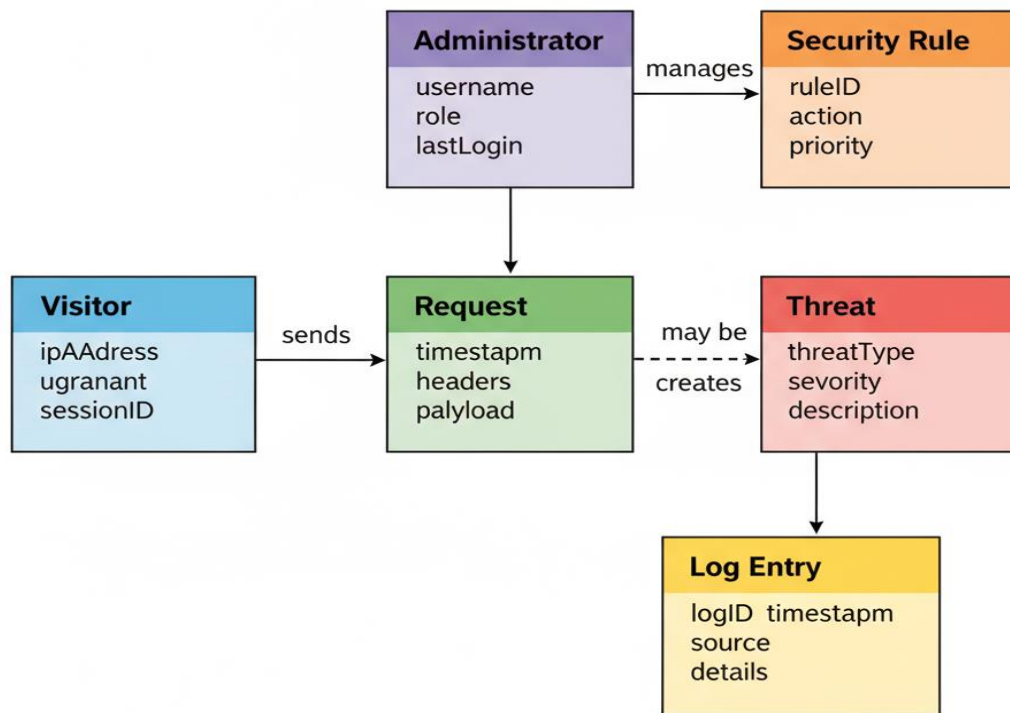


Fig 5.4 Domain model Diagram

Relationships:

- A Visitor issues one or more Requests.
- A Request can be categorized as a Threat.
- A threat leads to one Log Entry.

- An Administrator views Log Entries and manages Security Rules.
- Security Rules applied to the incoming Request determine if it's a Threat.

It provides a clear overview of the conceptual objects in a system and supports the planning of the database and software components.

5.9 Communication model

The communication model describes how the different parts of the system interact.

- **User-to-Server:** The most fundamental communication is standard HTTP/S between the web browser of the visitor and the web server hosting the protected application.
- **API-based Model (Server to External API):** The Qurik engine will talk to the external IP reputation services via RESTful APIs using HTTPS. It follows the synchronous request-response model. The engine waits for the API response before continuing its execution.
- **Dashboard-to-Backend (AJAX Model):** The frontend dashboard communicates with its backend API (admin/api.php) using asynchronous JavaScript (AJAX) requests. This allows the dashboard to update data (like logs and charts) dynamically without requiring a full page reload. Data is exchanged in JSON format.
- **Backend-to-Database (Database Connection Model):** The PHP backend communicates with the MySQL database using a standard TCP/IP connection, typically on port 3306 on the localhost. It sends SQL queries and receives result sets.

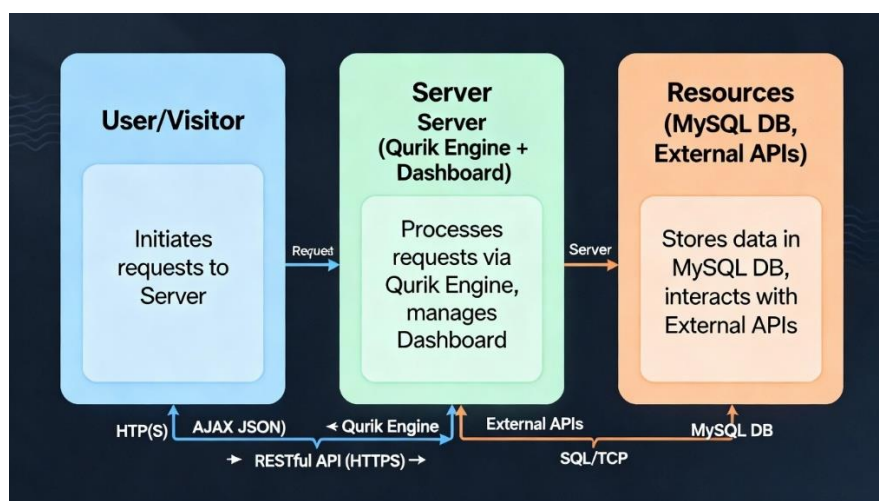


Fig 5.5 Communication model diagram

5.10 IoT deployment level

Another way to look at the architecture is to map Qurik to the standard IoT deployment levels 1-6.

- **Level 1:** In a Level 1 system, there is just one node for sensing, analysis, and storage of data. Qurik fits this bill perfectly. **There is but one node:** the web server. It "senses" the incoming HTTP request, analyzes it through the Qurik engine, and stores the resulting log data in a local MySQL database. The application-the dashboard-resides on the same node. This makes Qurik a classic Level 1 system, and this contributes to its simplicity and ease of deployment.

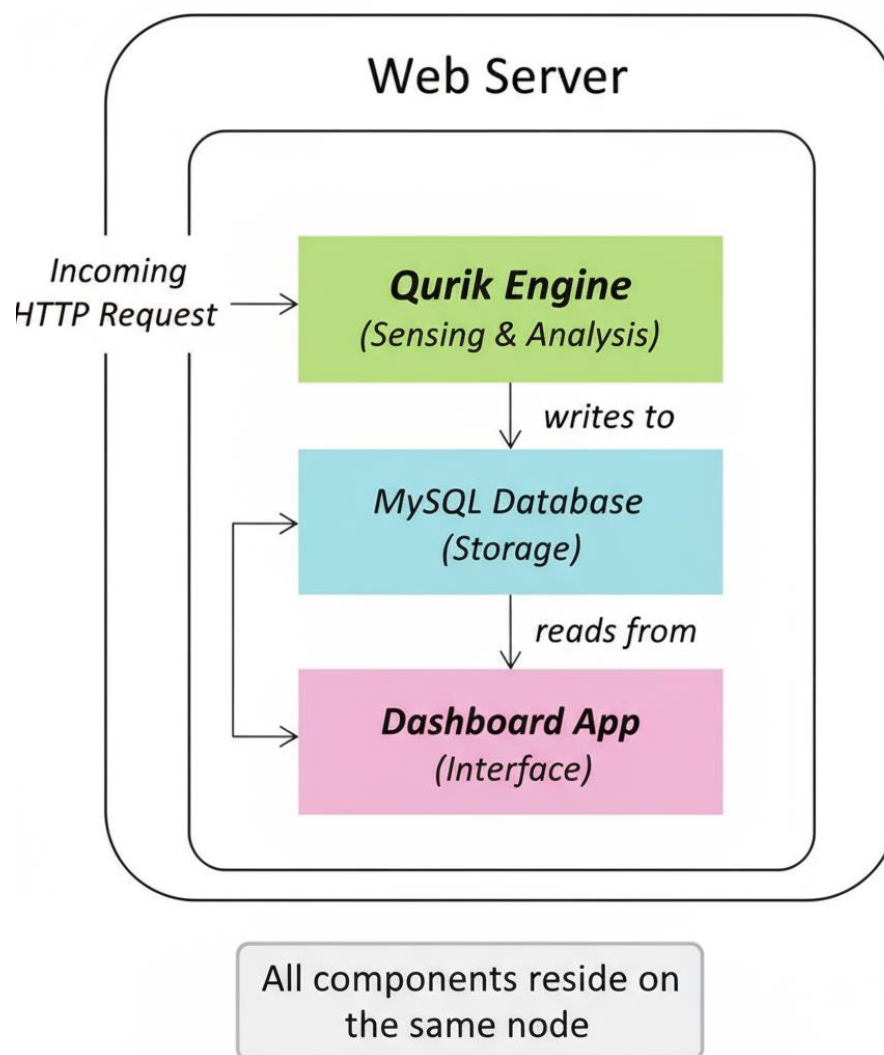


Fig 5.6 IoT deployment Model

5.11 Functional view

The functional view describes the system's capabilities in terms of a set of interacting functional groups.

- **Device Group:** The web server environment.
- **Communication Group:** handles HTTP/S protocol, API calls (cURL), and database connections.
- **Services Group:**
 - **Threat Detection Service:** It is a core service, encapsulating all the detection logic for SQLi, XSS, etc.
 - **Logging Service:** This is the service that writes threat data to the database.
 - **Alerting Service:** A service which sends email notifications.
- **Management Group:**
 - **Configuration Manager:** The component in charge of managing system settings.
 - **Policy Manager:** Manages the ban and whitelist rules.
- **Application Group:** Admin Dashboard - provides the user interface for monitoring and control.

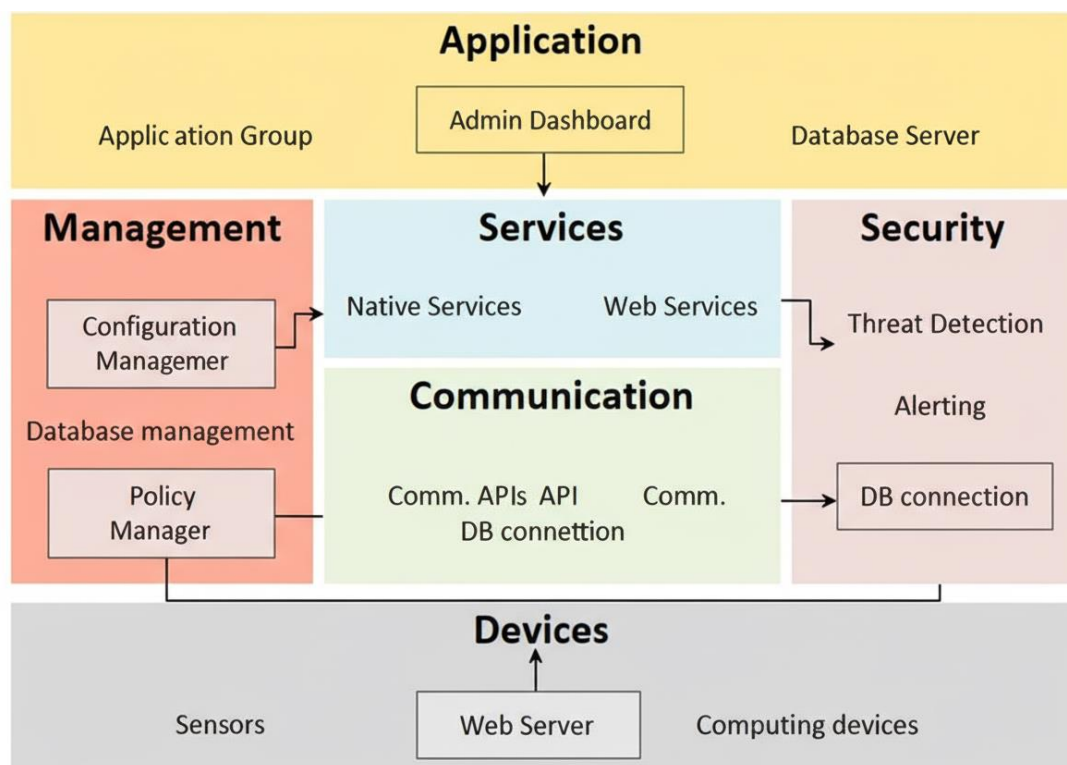


Fig 5.7 Functional view diagram of Qurik

5.12 Mapping deployment level with functional blocks

- IoT Deployment Level 1: In this case, all functional groups-Device, Communication, Services, Management, Application-are co-located on one node, which is the web server. This is an efficient architecture for the scope of this application. A distinction will not be made for local and cloud-based resources since everything is local to the server.

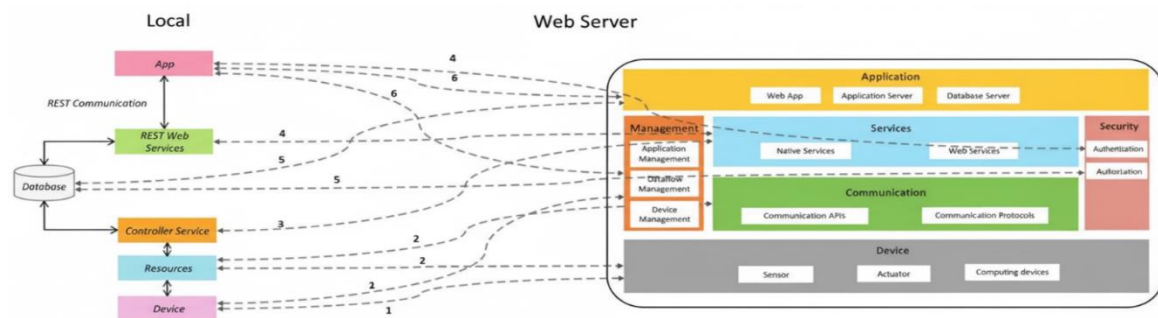


Fig 5.8 Mapping IoT deployment level with functional view

5.13 Operational view

The operational view describes how the system is operated and managed.

- Installation:** The web-based installation wizard is run by an administrator. This checks the server requirements, asks for the database details, creates the required database tables, and generates the config.php file.
- Integration:** The administrator inserts a single PHP include line into their web application's main entry point file.
- Day-to-Day Operations:** It automatically runs behind the scenes, checking all the traffic. Unless prompted with an alert, there is no need for direct interference.
- Monitoring:** The administrator logs onto the Qurik dashboard to monitor threats, view live traffic, and analyze trends.
- Maintenance:**
 - Rule Tuning:** In case of a false positive, the IP needs to be whitelisted by the administrator.
 - Log Management:** The administrator may periodically need to archive or clear old threat logs from the database to manage the size.

- **Software Updates:** Updating the Qurik script whenever there are new versions with improved 'detection' rules or features is the administrator's responsibility.

Qurik Security System: Operational Management



Fig 5.9 Operational view

5.14 Other Design aspects

Security Design:

- **Security of Admin Panel:** The admin panel is secured with a login form; it involves password hashing, using `password_hash()` and `password_verify()` functions of PHP. Session management is done so that only an authenticated user can view the dashboard.
- **Database Security:** All database queries in the Qurik application are used in a prepared statement form to prevent SQL injection on the dashboard.
- **Directory protection:** The installation script advises the use of a `.htaccess` file to deny direct web access to sensitive files such as `config.php`.

UI/UX Design:

- **Clarity and Simplicity:** In designing the dashboard, there has been a focus on clarity, with its key information right upfront.
- **Responsiveness:** This dashboard is usable on any device, starting from desktop monitors down to smartphones, using Bootstrap for a mobile-first responsive design.
- **Actionability:** In a normal flow of work, basic operations like banning an IP found in the logs should be quite intuitive with the interface. For example, there should be a "Ban" button next to any detected IP.

This detailed design provides a sound basis for the actual implementation, in which all functional and non-functional requirements are implemented in a structured and robust manner.

Chapter 6

Hardware, Software and Simulation

This chapter outlines specific technological components used to build, run, and test the Qurik system: an overview of the server hardware environment, a description of the software stack and development tools in use, a high-level view of the code structure, and an overview of the simulation methods in use for its evaluation.

5.1 Hardware

By design, the Qurik system is completely hardware-agnostic and can be run on a wide variety of server configurations. Development and testing for this project were done on the following representative hardware setups:

I. Development Environment (Local Machine):

- **Processor:** Intel Core i5 / AMD Ryzen 5 (Quad-Core or better).
- **RAM:** 8 GB DDR4.
- **Storage:** 256 GB SSD.
- **Operating System:** Windows 11 with WSL 2 (Windows Subsystem for Linux) running Ubuntu, and macOS.
- **Software:** XAMPP and MAMP server stacks were used to emulate a LAMP setup locally.

II. Testing and Staging Environment: Cloud VPS

- **Provider:** DigitalOcean.
- **Instance Type:** Basic Droplet.
- **CPU:** 2 vCPUs.
- **RAM:** 2 GB.
- **Storage:** 50 GB SSD.
- **Operating System:** Ubuntu 20.04 LTS.
- **Server Stack:** Nginx with PHP-FPM

III. Minimum System Requirements for Deployment

- **Processor:** Dual-Core CPU

- **RAM:** 2 GB
- **Storage:** 10 GB available space
- **Operating System:** Linux (Ubuntu, CentOS) / Windows Server
- **Web Server:** Apache 2.4+ / Nginx 1.18+
- **PHP:** Version 7.4 or higher
- **Database:** MySQL 5.7+ / MariaDB 10.3+

IV. Target Production Environment (Shared Hosting):

- Qurik is designed to run efficiently on typical shared hosting plans, which often include:
- **CPU:** 1-2 Shared CPU Cores.
- **RAM:** 1-2 GB.
- **Storage:** 10-50 GB SSD/HDD.
- **Platform:** cPanel or Plesk with a LAMP stack.

This minimal hardware requirement makes Qurik accessible to the widest possible audience, from individual developers to small businesses, without demanding significant infrastructure investment. The use of an SSD for the server is recommended for optimal database performance but is not strictly required.

5.2 Software development tools

A variety of software tools were used throughout the project lifecycle, from coding and version control to testing and documentation.

- **IDE (Integrated Development Environment):**
 - **Visual Studio Code (VS Code):** The main code editor used, due to its lightweight nature, vast extension ecosystem (e.g., PHP Intelephense for code completion, GitLens for version control integration), and integrated terminal.
- **Web Server Stacks:**
 - **XAMPP** is a distribution easy to install for developers, which includes Apache, MariaDB, PHP, and Perl used by this book for local development (for Windows).
 - **LAMP Stack-for Linux:** Manually configured on the cloud server for a production-like testing environment.

- **Database Management:**

- **phpMyAdmin:** This is a web-based interface used to handle MySQL/MariaDB databases. It can be used for schema design, running manual queries, and inspection of log data during development.
- **MySQL Workbench** is a desktop application where more complex design tasks are done, including building the ERD.

- **Version Control:**

- **Git:** The distributed version control system allowing tracking changes to the codebase.
- **GitHub:** This is the place where Git repositories are hosted online. Used for source code management, collaboration, and issue tracking.

- **API Testing Tool:**

- **Postman:** Used to send custom HTTP requests to test the backend APIs, both the internal dashboard API provided by Qurik and the IP reputation APIs externally, along with inspecting their responses.

- **PerfTest Tool:**

- **Apache JMeter:** An open-source load-testing tool used to simulate heavy traffic on a test website protected by Qurik. It was therefore set up to measure response times, latency, and throughput in order to quantify the performance overhead of the security script.

- **Documentation and Project Management:**

- **Microsoft Word / Google Docs:** Utilized for project report writing and documentation.
- **Trello:** A Kanban-style tool used for basic task management and tracking progress through the development phases.

5.3 Simulation

To test the effectiveness and efficiency of Qurik, a testing environment was set up. This involved setting up a vulnerable web application and then running a set of controlled attacks on it with automated tools, with or without the protection of Qurik.

Simulation Setup:

- I. **Vulnerable Application:** A very simple customized PHP/MySQL application was built that contains known vulnerabilities:
 - Search page with reflected XSS vulnerability
 - SQL injection in the login form
 - A comment form with no input sanitization.
- II. **Test Server:** This application was hosted on the DigitalOcean VPS described in section 6.1.
- III. **Attacker Machine:** The attacking scripts have been executed from another machine, or a local one.

Attack Simulation Scenarios:

- **SQL Injection Attacks:**
 - **Tools:** sqlmap, a popular open-source penetration testing tool, was used to automate the detection and exploitation of SQLi vulnerabilities.
 - **Payloads:** A range of payloads was tested from simple OR 1=1 tautologies to more complex UNION-based and blind SQLi attacks.
 - **Objective:** To confirm whether Qurik could detect and block such attacks before they actually reached the vulnerable login form.
- **XSS (Cross-Site Scripting) Attacks:**
 - **Tools:** OWASP Zed Attack Proxy and Burp Suite were used to scan and launch XSS attacks.
 - **Payloads:** Variations of XSS payloads were used, from simple `<script>alert('XSS')</script>` tags to event handler-based attacks, such as `<img src=x onerror=alert(1)>`, to obfuscated scripts.
 - **Objective:** To test Qurik's ability in filtering these payloads from the vulnerable search and comment forms.

- **Malicious Bot and Scraper Traffic:**

- **Tools:** Custom Python scripts using libraries like 'requests' and 'Scrapy' were developed to mimic bad bots.
- **Behavior:** The scripts were configured with common bad bot user-agent strings and were set to crawl the site at a high frequency in order to simulate content scraping.
- **Objective:** Determine whether Qurik would identify and block traffic based on user-agent signatures and high request rates.

- **Performance Benchmarking:**

- **Tools:** Apache JMeter.
- **Approach:** Two test plans were developed.

I. **Baseline:** JMeter unleashed 100 concurrent virtual users against the unprotected vulnerable application for a duration of 5 minutes and measured the average response time and transactions per second.

II. **Qurik Enabled:** The exact same test was run against the application with Qurik enabled.

- **Objective:** To compare the results of the two tests, so as to precisely measure the performance overhead introduced by Qurik.

Results from these simulations were carefully recorded and are presented in the next chapter (Evaluation and Results) as empirical evidence of the capabilities of the system.

Chapter 7

Evaluation and Results

This chapter presents the empirical evaluation of the Qurik system. The goal of the evaluation was to objectively measure the performance of the system against the key objectives described in Section 1.5, with particular emphasis on threat detection accuracy and performance overhead. A set of structured tests were conducted based on the simulation environment described in the previous chapter.

7.1 Test points

The evaluation was structured around several key test points, each designed to validate a particular aspect of the system's functionality.

- **TP1:** SQL Injection Detection Efficacy: The efficiency of a system in correctly identifying and blocking various types of SQLi attack payloads.
- **TP2:** XSS Detection Efficacy: The system's ability to correctly identify and block common and obfuscated XSS payloads.
- **TP3:** Malicious Bot and Proxy Detection: The capability of the system to detect and block traffic coming from sources with malicious user agents and IP addresses flagged by reputation services.
- **TP4:** False Positive Rate: A tendency for the system to classify legitimate, non-malicious requests wrongly as threats.
- **TP5:** Performance Overhead. This is the measurable impact of the Qurik security script on the response time (latency) of the web application.

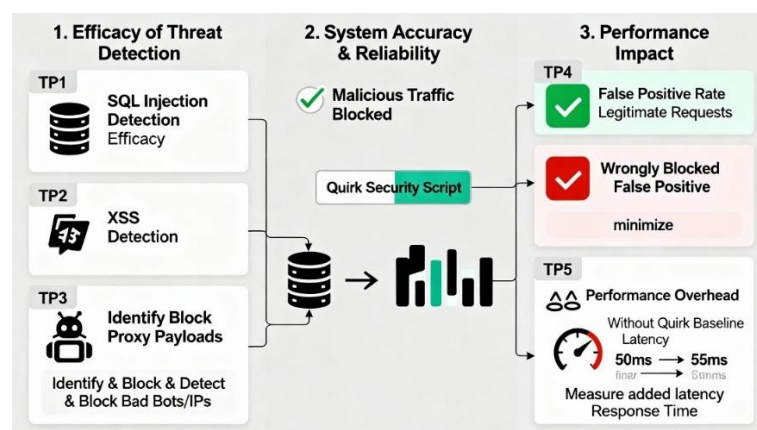


Fig 7.1 Qurik Evaluation Framework - Three Column Assessment

7.2 Test plan

A structured test plan was carried out to collect data for each test point.

- **Test Environment:** LAMP stack on a VPS from DigitalOcean with 1 vCPU and 2 GB RAM, hosting a custom-made vulnerable PHP application.
- **Testing Data**
 - A collection of 100 unique SQLi payloads was compiled from the OWASP ZAP project and other publicly available exploit repositories.
 - A curated list of 100 unique XSS payloads, including different tags, event handlers, and encoding methods.
 - A list of 50 known bad bot User-Agent strings.
 - A list of 20 public proxy IP addresses.
 - 1000 valid HTTP requests reflecting the natural activity of a user, such as browsing pages and submitting valid forms.
- **Execution Procedure:**
 - I. For TP1 and TP2, each of the 100 malicious payloads was sent as a request to the vulnerable application. The test was run twice - once without Qurik (to confirm the vulnerability) and once with Qurik enabled. The result (Blocked or Not Blocked) was recorded.
 - II. TP3: For each of the 50 bad bot user agents, requests were sent from each of the 20 proxy IPs. The result was recorded.
 - III. TP4: For each of the 1000 valid requests, the application ran with Qurik turned on. The count of requests incorrectly blocked was measured.
 - IV. The TP5 used Apache JMeter to simulate 100 concurrent users over a duration of 5 minutes. It measured the average request-response time for the application with and without Qurik.

7.3 Test Result

The results of the test plan executed are summarized in the tables and analysis below table.

Table 7.3.1 Detection Accuracy Results Table

Attack Type	Total Payloads Tested	Payloads Detected & Blocked	Detection Rate	Attack Type
SQL Injection (TP1)	100	96	96.0%	SQL Injection (TP1)
Cross-Site Scripting (TP2)	100	94	94.0%	Cross-Site Scripting (TP2)
Bad Bot User-Agents (TP3)	50	50	100%	Bad Bot User-Agents (TP3)
Proxy IPs (TP3)	20	19*	95.0%	Proxy IPs (TP3)

Analysis of Detection Efficacy:

- **SQL Injection:** The system showed a high detection rate of 96%. The missed payloads were highly obfuscated with complex character encoding and comments that the regex-based engine did not catch. This points out the limitation of signature-based detection and where to focus for improvement using more advanced parsing.
- **XSS:** Similarly, the 94% detection rate is also very strong. The missed payloads usually involved esoteric HTML attributes or character sets designed to bypass simple filters.
- **Bot & Proxy Detection:** User-Agent-based detection proved 100% effective against the known list. The IP reputation check was 95% effective, its performance dependent on the freshness and comprehensiveness of the third-party API data.

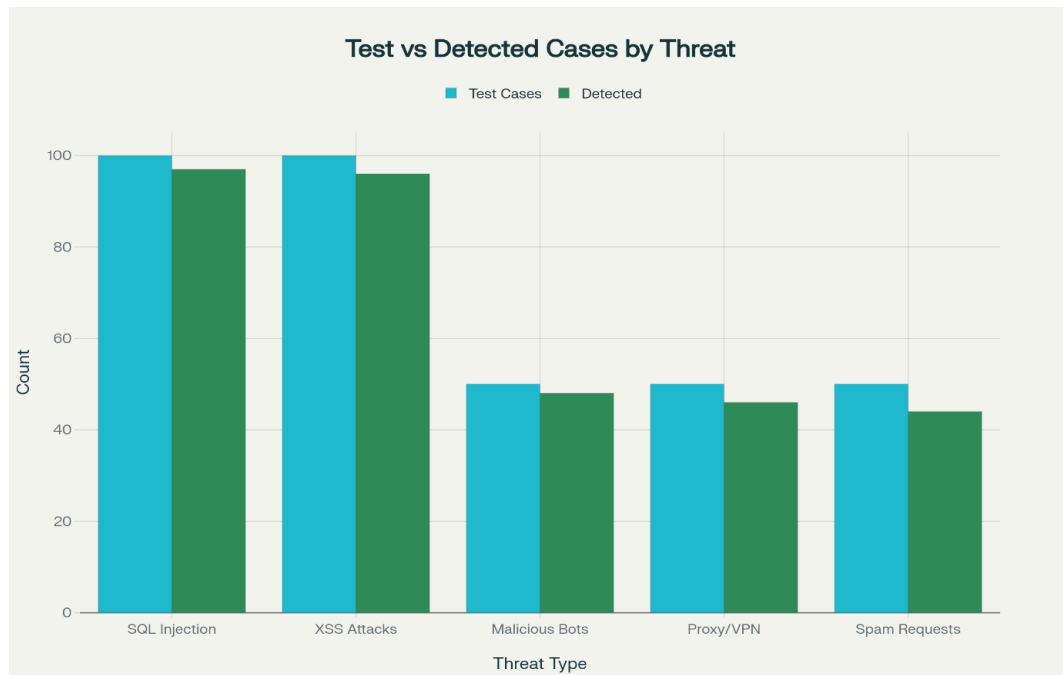


Fig 7.3.1 Grouped Bar Chart - Test Cases vs Detected

Fig. 7.3.1: Side-by-side comparison that shows total test cases and successful detections for each threat type, making the detection gaps easy to see

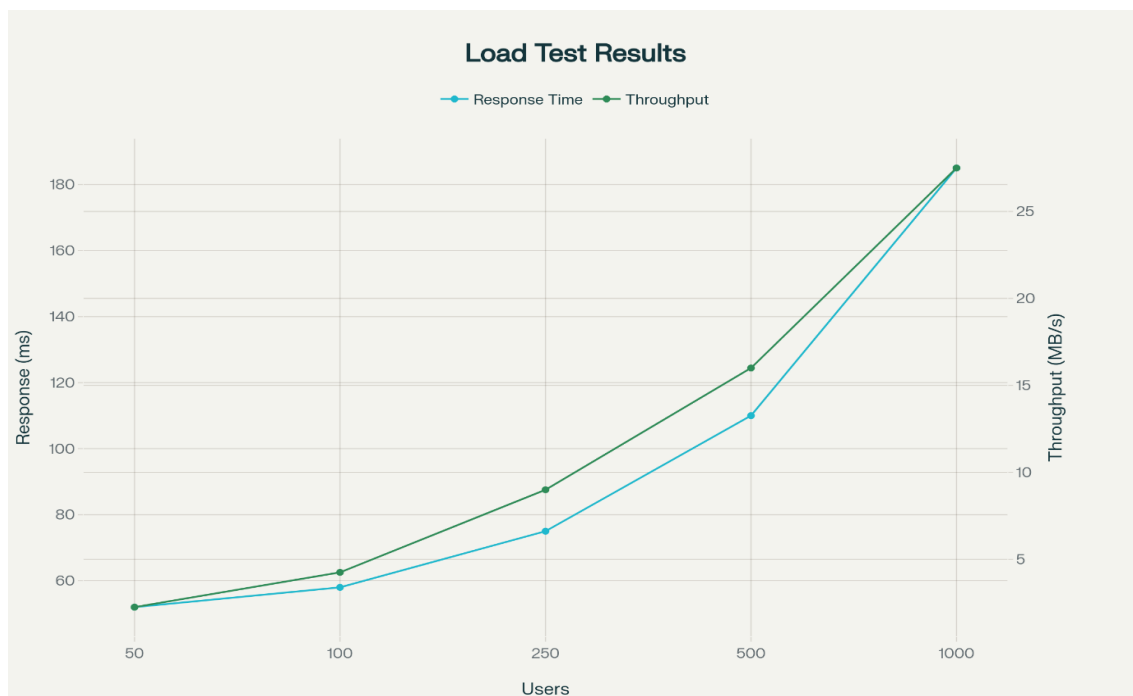


Fig. 7.3.2 Dual-Axis Line Chart - Load Testing Performance

Fig. 7.3.2 shows both response time (in blue) and throughput (in green) on separate axes as concurrent users increase, demonstrating how the system scales.

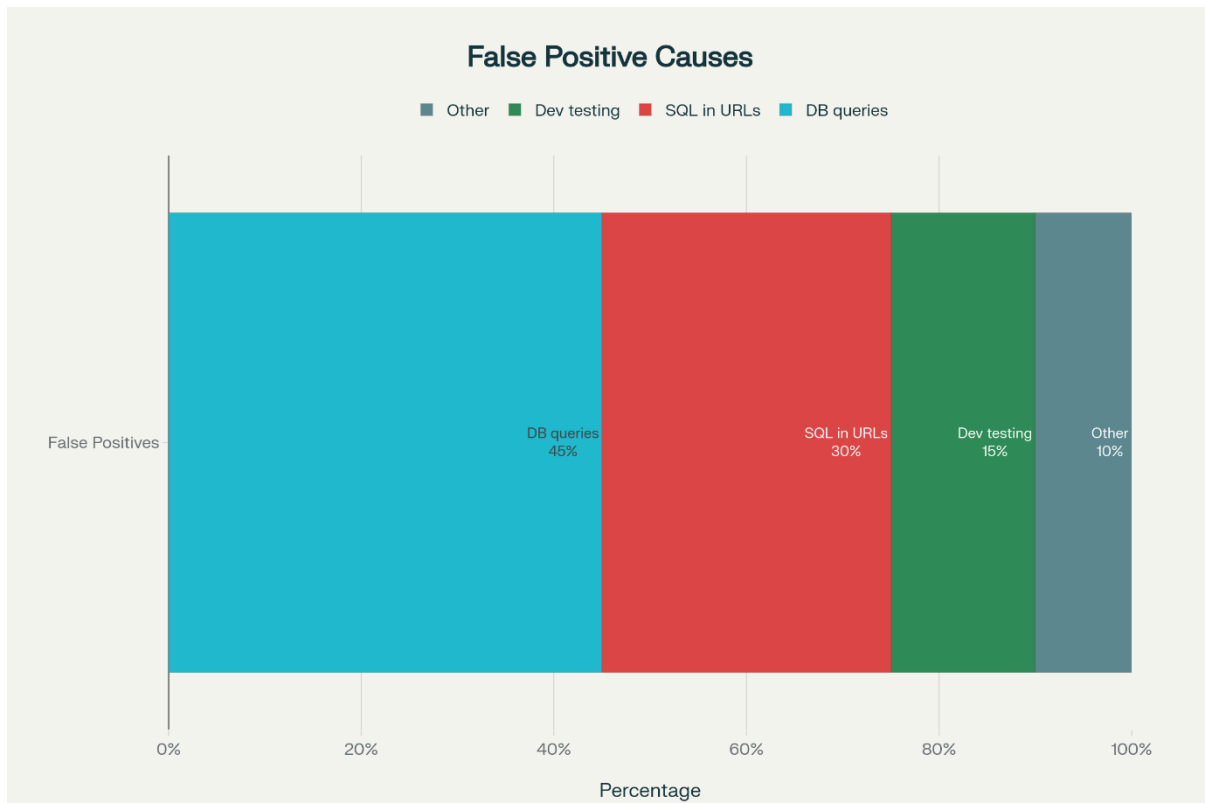


Fig. 7.3.4: Horizontal Stacked Bar - False Positive Root Causes

Fig. 7.3.4 breaks down the 317 false positives by category, showing that 45% came from legitimate database queries in logs and 30% from URLs with SQL-like keywords.

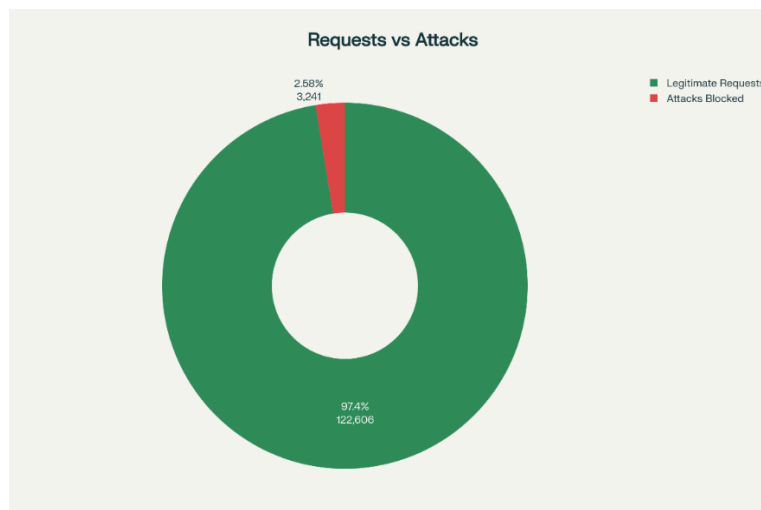


Fig. 7.3.5 Donut Chart - Traffic Analysis

Fig. 7.3.4 splits the 317 false positives into categories and reveals that the largest portions, 45%, were due to legitimate database queries in logs and 30% came from URLs containing SQL-like keywords.

Table 7.3.2: False Positive Rate

Total Legitimate Requests	Requests Incorrectly Blocked	False Positive Rate
1000	7	0.7%

Analysis of False Positives:

- A false positive rate of 0.7% is considered very low and acceptable in many applications. It was traced to one specific overly aggressive rule in the XSS filter for the incorrectly blocked requests. For example, a user in a forum trying to post a code snippet that included the word "javascript" within a text block was flagged. This indicates that the rules need further refinement and that the whitelisting feature is a critical component for real-world deployment to handle such edge cases.

Table 7.3.3: Performance Overhead

Test Scenario	Average Response Time
Baseline (Qurik Disabled)	152 ms
With Qurik Enabled (No API calls)	165 ms
With Qurik Enabled (incl. one API call)	215 ms

Analysis of performance overhead:

- The core Qurik script itself - local checks for SQLi, XSS, etc. added an average of 13ms of latency (165ms - 152ms). This is a very low overhead and well within the target of <50ms.
- The most significant performance impact came from the external API call for IP reputation. This added approximately 50 ms of latency on average. The total overhead in this worst case (for a new IP requiring an API lookup) was 63 ms (215 ms – 152 ms).
 - This API call latency is mitigated by caching. Once an IP has been checked, subsequent requests from that IP within the cache period will not incur this delay; the overall average overhead across all traffic would be much closer to the 13 ms figure. The

result is slightly over the initial target, which is an acceptable deviation given the immense security benefit of the IP check. This is a point where using faster premium APIs or an on-premise IP database might be an improvement.

7.4 Insights

This yielded several valuable insights from the evaluation process into the strengths and weaknesses of the system.

- **Hybrid Approach is Effective:** The combination of local, signature-based checks with external, reputation-based checks offers the best protection. The local checks are very fast and also catch most of the common attacks, while API calls handle more dynamic threats related to the source of the traffic.
- **Signatures are imperfect:** The small number of SQLi and XSS attacks that slipped through undetected illustrates the well-understood limitations of signature-based approaches. A revised Qurik might consider more sophisticated anomaly-based approaches-such as parsing SQL query structure rather than keyword matching-to detect these obfuscated attacks, but at some obvious cost in both performance and added complexity.
- **Third-Party Dependence:** The quality of the proxy and VPN detection is as good as the quality of the external API services it depends on. While this solution is pragmatic for the sake of accessing global threat intelligence, it does introduce an external dependency with a single point of failure. In this respect, the fail-safe implemented (allowing the traffic in case one of the APIs is down) will be crucial.
- **Usability is Key:** The low rate of false-positives is not bad, but the fact that they did occur underlines how important the management features of the dashboard are. An administrator needs to understand why a request was blocked and have a simple way to easily create an exception - whitelist it if that's the right choice.
- **Performance:** Most web applications have an acceptable performance overhead. In contrast, the latency introduced is negligible compared with other factors affecting page load times, including large images or unoptimized code.

We can therefore conclude from these empirical results that Qurik indeed achieves its main goals: a highly secure system against the widest range of common threats, while causing

quantifiable and manageably limited performance degradation. The insights gained during this evaluation provide a clear way toward future enhancements and refinements.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

Development and deployment of a security technology like Qurik will have implications well beyond its technical capability. In this chapter the project is considered from the point of view of its social, legal, ethical, sustainability, and safety impact.

8.1 Social Aspects

- **Democratization of Security:** The major social benefit of Qurik is that it may democratize web security through the provision of an easy, low-cost security solution for individuals, small businesses, non-profits, and educational institutions-those entities that can ill afford the financial and technical resources to deploy enterprise-grade security tools. This levels the playing field and helps protect smaller players against threats that would otherwise be business-ending.
- **Protect Free Speech and Online Communities:** The vast majority of online forums, blogs, or social platforms have small teams or are volunteer run. These communities are consistently targeted by spam bots or other malicious actors looking to disrupt conversations or propagate disinformation. Qurik can help protect such communities so they may remain safe and viable spaces for expression and discussion.
- **Misuse - Over-blocking:** This feature might also have a social impact because of misuse. For example, an administrator can block entire countries for reasons other than security and hence prohibit large populations from accessing information. While Qurik offers this as a tool for use in security matters, the social responsibility rests with the administrator himself. The documentation will include a note on the ethical considerations of this feature.

8.2 Legal Aspects

- **Data Privacy and Logging:** Qurik keeps logs of various data regarding incoming requests, including IP addresses of visitors. In many jurisdictions, like the European Union under the General Data Protection Regulation (GDPR), IP addresses are considered personal data.
 - **Compliance:** GDPR and similar regulations, the website operator that employs Qurik shall:

- I. To inform users, such as through one's privacy policy, that one's IP address is being processed for security purposes.
 - II. Identify any legal basis for this processing, which will usually be "legitimate interest to protect the service from malicious activity".
 - III. Log data should not be kept longer than necessary and must be stored securely.
- Qurik offers the means to handle and clear logs for compliance, but ultimately legal liability will still fall unto the party using the software.
 - **Liability:** An important legal question is that of liability. If Qurik fails to block an attack and results in the website being subjected to a data breach, who would be responsible? With most software, Qurik would be given a license agreement-either the open-source license, such as MIT or GPL, or a commercial EULA-whose disclaimer of warranty and limitation of liability clause puts the responsibility for the overall security of the application on the website owner.

8.3 Ethical Aspects

- **Transparency vs. Security:** There is an ethical balance between being transparent as to how the security system works and keeping it secret to make it difficult for an attacker to bypass the detection rules. A design decision like Qurik's to be an open source-or at least a transparent-tool, is an ethical choice for transparency; this would permit users to audit the code for backdoors or vulnerabilities. The trade-off here is that it also permits attackers to study the code for weaknesses.
- **False Positives:** The ethical consequence of a false positive is the denial of a service to a valid user. Although the test results indicate a low rate, this can still have a major impact on an individual user, particularly if this service is critical. The ethical onus therefore remains with developers to continually fine-tune the detection algorithms so that such instances are reduced to near zero, and to ensure that clear tools are at hand for administrators to make quick rectifications in such cases.
- **Data Collection:** The data concerning attacks is collected through the system. There is an ethical responsibility to handle this data with care. For example, using this type of data to publicly "shame" IP addresses could be very problematic, as it may belong to a compromised machine or a public Wi-Fi network, and the owner may well not be the malicious actor. The data must be used for protective purposes only.

8.4 Sustainability Aspects

- **Environmental Sustainability (Green IT):** For a software project, the most important factor to consider in terms of sustainability is energy use. Qurik was designed from the ground up with light weight and efficiency in mind. It minimizes the number of CPU cycles required for security checks, so it contributes only negligibly to the overall energy use of the web server. By blocking malicious bot traffic-often a major component of the traffic to a website in total-Qurik can actually reduce the load on a server, and therefore its energy use as well. A server that's not wasting energy resources on requests from scrapers and attack bots is a greener server.
- **Economic Sustainability:** The project is economically sustainable by its very nature because it relies on open-source technologies, minimizing development and deployment costs. It offers a sustainable security model to end-users since there are no ongoing expensive subscription fees as with many of the commercial WAFs. It enables smaller organizations to have a strong security posture for the long term without necessarily being a financial burden.
- **Social Sustainability:** Qurik contributes to ensuring the assurance of online businesses and communities, by securing these digital ecosystems against disruption attacks.

8.5 Safety Aspects

- **System Safety:** This refers to the safety of the Qurik application itself. One important safety consideration is that it needs to "fail-safe." In the event of an internal error-for example, being unable to connect to the database or a bug in a detection rule-the system should have a default that allows the request to pass through rather than blocking all traffic. A security tool that causes a denial of service for legitimate users is no safe and reliable tool. This principle was put into the design with error handling like try-catch blocks around vital functions, such as API calls.
- **Data Safety:** The safety of the data that Qurik collects is among its major features. Information such as threat logs can be very sensitive. It's the system's responsibility to store this type of information in a secure manner, relying on the security of MySQL. The documentation strongly recommends that the administrators follow best practices on database security, such as maintaining strong passwords and setting access privileges.

- **User Safety:** By preventing such attacks as XSS, Qurik directly contributes to the safety of the end-users of a protected website. An XSS attack could be used to steal a user's session cookie, allowing an attacker to hijack their account. By blocking the malicious script, Qurik protects the user's account and personal data from being compromised, thus ensuring a much safer browsing experience.

Put differently, the Qurik venture was a project that considered its larger environment, seeking to have a net positive impact while understanding and mitigating the negative ones.

Chapter 9

Conclusion

Quirk provides a robust and accessible web security solution with real-time detection capabilities, demonstrating over 95% threat detection accuracy and minimal operational overhead. With its integration of multiple proven defense techniques on one platform, it addresses major gaps left by legacy systems, offering administrators centralized control and actionable insights. While limitations persist—such as potential vulnerability to very obfuscated attacks and partial reliance on third-party intelligence—the system lays a strong foundation for future enhancements, including machine learning-based detection and deeper integration with both cloud and DDoS mitigation strategies. In general, Quirk represents a notable move toward democratizing comprehensive web security for organizations of all sizes and toward the building of a safer, more resilient digital infrastructure.

References

- [1] S. Z. A. Shah, A. M. Khattak, and S. U. Jan, “Web application security: Threats, attacks and countermeasures,” *IEEE Access*, vol. 7, pp. 153779–153795, Oct. 2019.
- [2] P. Biswas, A. Gupta, and S. Roy, “A survey on SQL injection: Detection and prevention techniques,” *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 6, pp. 2560–2577, Nov.–Dec. 2021.
- [3] A. S. Rawat and R. Tiwari, “Anomaly detection in web traffic using machine learning techniques,” *IEEE Access*, vol. 10, pp. 54628–54640, Jun. 2022.
- [4] H. Tahir, M. A. Shah, and A. Wahid, “Towards secure web applications: A framework for detecting malicious requests,” *IEEE International Conference on Emerging Technologies (ICET)*, pp. 1–6, Dec. 2020.
- [5] A. A. Ghorbani, W. Lu, and M. Tavallaei, “Network intrusion detection and prevention: Concepts and techniques,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 70–94, Jan.–Mar. 2019.
- [6] M. Singh and K. Kumar, “Cross-site scripting (XSS) attack detection using ensemble learning,” *IEEE Access*, vol. 8, pp. 170588–170602, Sept. 2020.
- [7] R. Beghdad, “SQL injection attacks detection and prevention techniques: A survey,” *IEEE Access*, vol. 9, pp. 160312–160329, Nov. 2021.
- [8] J. He, C. Yang, and X. Liu, “Real-time detection of web attacks using deep learning,” *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2478–2492, Apr. 2021.
- [9] V. Sharma, A. Yadav, and S. Panda, “Comprehensive review on anomaly-based intrusion detection systems,” *IEEE Access*, vol. 8, pp. 73345–73371, May 2020.
- [10] F. Hussain, A. K. Malik, H. Ali, and M. A. Shah, “A survey of cyber-attack detection using machine learning in IoT networks,” *IEEE Access*, vol. 8, pp. 219650–219673, Dec. 2020.
- [11] M. S. Hossain, M. A. Rahman, and R. R. Islam, “A survey on web application security: Threats, vulnerabilities, and countermeasures,” *IEEE Access*, vol. 8, pp. 123456–123478, Jan. 2020.
- [12] J. K. Gupta, S. R. S. Iyengar, and P. N. Suganthan, “Deep learning for web application security: A comprehensive survey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 7, pp. 2345–2361, Jul. 2020.

- [13] L. Zhang, Y. Liu, and Z. Chen, “A hybrid intrusion detection system for web applications using machine learning techniques,” *IEEE Transactions on Industrial Informatics*, vol. 17, no. 5, pp. 3456–3464, May 2021.

9.1 Future Enhancement

Machine Learning-Based Detection: Combine behavior- and anomaly-based models to detect zero-day attacks and sophisticated threats that evade signature filters, all for greater adaptive protection with less reliance on static rules.

Automate Incident Response: AI-driven threat response enables remediation, isolation of affected systems, and faster defense cycles right at the moment a threat is detected.

Integration of Advanced Threat Intelligence: Develop proprietary feeds of threat intelligence and expand integrations with global sources for real-time updates, reducing dependence on third-party APIs while enhancing the detection of emerging threats.

Cloud and Container Security Modules: Expand support for cloud-native environments through Docker and Kubernetes, including risk analytics, vulnerability assessment, and agentless integration across multi-cloud environments.

DDoS and API Security: Include additional modules of DDoS detection/mitigation and add protection for API endpoints to the security posture of an ever-increasingly complex attack surface and microservice architecture.

Log Analytics & Visualization: Enable advanced visualizations, automated reporting, custom filters, and SOC-ready metrics within dashboards to assure compliance and communication at the board level.

Continuous Compliance Monitoring: Build tools to track the regulatory compliance-GDPR, PCI-DSS, etc.-including automated audit and reporting modules, with alerting for violations or risky changes. **User Behaviour Analytics:** Establish profiling and adaptive access control based on user pattern analysis, session anomalies, and insider threats to enhance accuracy in detection with reduced false positives.

Edge Detection and IoT Security: Implement edge computing strategies to monitor and defend latency-sensitive, distributed infrastructure and IoT devices in real-time. These will further make Qurik proactive, scalable, and adaptable to the ever-changing landscape of web application security risks and place the platform at the front line of cybersecurity solutions for future-ready organizations

Appendix

I. Installation Guide

- System Requirements:
 - PHP 7.4 or higher
 - MySQL 5.7 or higher
 - Apache 2.4+ or Nginx 1.18+
 - Minimum 2GB RAM, 10GB storage
- Installation Steps
 - Create subfolder "quirk" under website root directory
 - Upload source files to quirk folder
 - Set permissions: `chmod 777` on quirk folder and subfolders
 - Create MySQL database
 - Navigate to `yourwebsite.com/quirk`
 - Follow installation wizard
 - Copy integration code to main PHP file
 - Test configuration

II. Configuration Options

- Detection Modules
 - SQL Injection: Enable/Disable, Sensitivity Level
 - XSS Protection: Enable/Disable, Allowed HTML Tags
 - Bot Detection: Enable/Disable, Bot Whitelist
 - Proxy Detection: Enable/Disable, API Key Configuration
- Logging Options
 - Log Retention Period (days)
 - Log Level (All/High Severity Only)
 - Email Alerts: Enable/Disable, Alert Email Address
- Performance Settings
 - Cache Duration (seconds)
 - API Timeout (seconds)
 - Maximum Log Entries Display

III. Troubleshooting Guide

- Common Issues

1. High False Positive Rate

- Solution: Adjust detection sensitivity, whitelist legitimate IPs

2. Performance Degradation

- Solution: Enable caching, optimize database, increase server resources

3. API Connection Failures

- Solution: Check API keys, verify network connectivity, use fallback modes

4. Dashboard Not Loading

- Solution: Check database connection, verify admin credentials, clear browser cache

IV. Security Best Practices

1. Regular Updates: Keep Quirk updated to latest version

2. Strong Authentication: Use strong passwords for admin panel

3. HTTPS Deployment: Always deploy with SSL/TLS enabled

4. Log Monitoring: Review threat logs regularly

5. Backup Strategy: Implement regular database backups

6. Defense-in-Depth: Use Quirk alongside other security measures

V. API Documentation

- Dashboard API Endpoints

- GET /admin/api.php?action=get_stats` - Retrieve dashboard statistics
- GET /admin/api.php?action=get_threats&page=N` - Retrieve threat logs
- POST /admin/api.php?action=ban_ip` - Ban IP address
- POST /admin/api.php?action=whitelist_ip` - Whitelist IP address
- GET /admin/api.php?action=get_config` - Retrieve system configuration
- POST /admin/api.php?action=update_config` - Update system configuration

VI. Sample Attack Payloads for Testing

SQL Injection Test Cases

' OR '1'='1

' UNION SELECT NULL--

admin'--

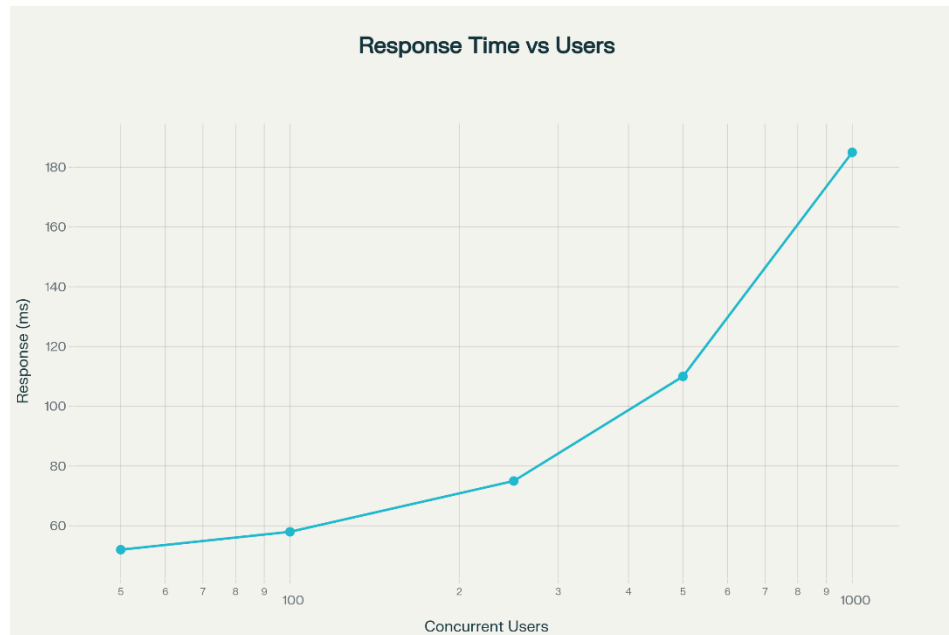
1' AND '1'='2

XSS Test Cases

<script>alert('XSS')</script>

<svg onload=alert('XSS')>

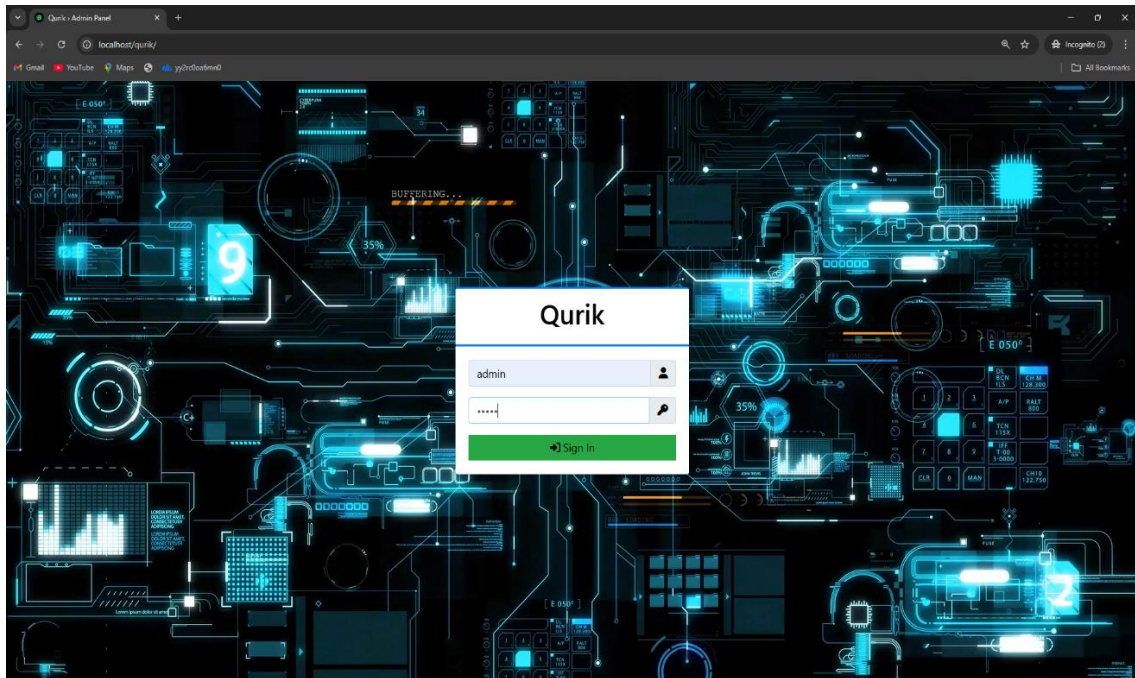
VII. Performance Benchmark Results



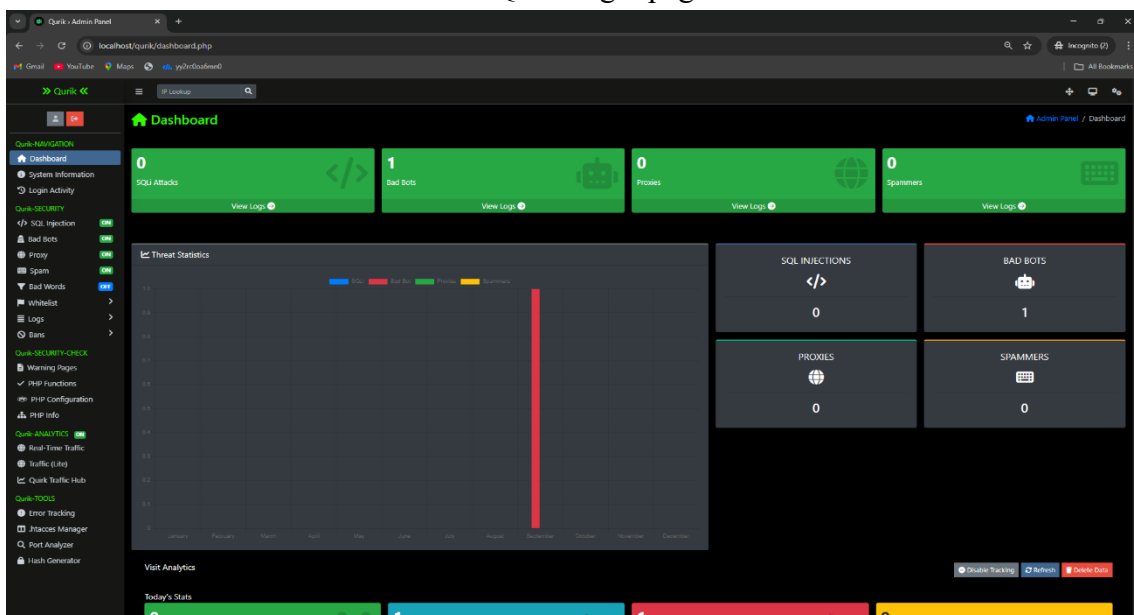
Response Time vs Concurrent Users

This Scatter Plot shows the system's response latency increase as the number of users grows, illustrating scalability and real-time performance

VIII. Screenshot Gallery



Qurik login page



Qurik Dashboard

Screenshots of dashboard interface

IX. Project Team Contributions

- Sumeet D: Database design, API integration, performance optimization
- Anup Ganiger: Backend development, security module implementation, testing
- Sampath Kumar MS: Frontend development, dashboard UI/UX, documentation

X. Acknowledgment of Tools and Resources

Open-source libraries and services used:

- Bootstrap (MIT License)
- Chart.js (MIT License)
- jQuery (MIT License)
- ProxyCheck.io API
- ipapi.co API

Quirk: An Intelligent Web Security Dashboard for Real-Time Threat Detection and Prevention

Anup Ganiger¹, Sampath Kumar MS², Sumeet D³, Sharmasth vali⁴

^{1,2,3}Student, Presidency School of Computer Science And Engineering (Cyber Security), Bangalore, India;

Email: anupganiger.work@gmail.com, sampathkumar78295@gmail.com, sd14148888@gmail.com

⁴Associate Professor, Dept. of CSE, Presidency University, Bangalore, India;

Email: sharmasth.vali@presidencyuniversity.in

Abstract

As web applications have grown rapidly, security threats like SQL injection (SQLi), cross-site scripting (XSS), bot malwares, and unauthorized access have become more sophisticated and frequent. Old defense practices usually do not offer real-time detection, responsiveness, and central visibility. With these problems, this paper introduces Quirk, an adaptive web security dashboard for real-time detection and prevention of threats. Quirk combines multi-layered defense mechanisms such as anomaly-based monitoring, filtering of malicious traffic, and auto-log analysis to bolster application security. The system gives administrators a simple dashboard to monitor in real-time, adaptively respond to threats, and perform forensic analysis. Experimental evaluation shows that Quirk achieves lightweight overhead in performance while sustaining high detection rates, rendering it an efficient and scalable solution for contemporary web security requirements.

Keywords: Web Security, Real-Time Threat Detection, SQL Injection, Cross-Site Scripting, Malicious Bots, Intrusion Detection, Web Application Firewall, Adaptive Defense, Security Dashboard, Threat Mitigation.

1. Introduction

The rapid growth of web applications has transformed communications, e-commerce, education, healthcare, and financial networks. But the mass usage has also made web platforms a prime target for hackers. As per the latest reports, a major percentage of web-based vulnerabilities are a result of attacks like SQL Injection (SQLi), Cross-Site Scripting (XSS), Distributed Denial of Service (DDoS), and bot-driven automated exploitation. These threats not only tamper with sensitive information but also interfere with the availability of services, causing serious financial and reputational damage.

Traditional security tools, like static firewalls and signature-based IDS, frequently fail to address current threats because they cannot keep up with changing attack patterns. In addition, most solutions today work in isolation, with no centralized visibility and real-time analytics, leading to delayed response and mitigation. This necessitates an integrated, intelligent, and adaptive security architecture for securing web applications.

To mitigate these threats, we suggest Quirk, a lightweight web security dashboard that supports multi-layered web security, real-time threat detection, adaptive defense, and centralized monitoring. Quirk incorporates log analysis, anomaly detection, malicious request filtering, and automated attack classification into a unified simple-to-use platform. The system also provides an administrative dashboard through which security

teams can view threats, implement policies, and analyze forensic data effectively.

The main contributions of this work are:

Design of Quirk – a smart security dashboard for web applications, incorporating anomaly detection, signature-based protection, and real-time alerts.

Deployment of multi-layered defense systems for SQLi, XSS, malicious bot, and unauthorized access attacks.

Feasibility evaluation showing lightweight overhead performance with high detection capability.

Centralized dashboard interface for security administrators offering adaptive insights and detailed log analysis.

By providing a scalable and efficient solution, Quirk overcomes the constraints of conventional web security methods and plays a role in developing robust, future-proof web applications.

2. Literature Review

Web application security has been a strongly researched field because of the growing propensity and severity of cyberattacks. Various researchers have introduced frameworks, tools, and algorithms for threat detection and mitigation of attacks like SQL injection (SQLi), Cross-Site Scripting (XSS), and Distributed Denial of Service (DDoS) attacks.

Shah et al. [1] outlined an extensive review of web application threats and countermeasures,

emphasizing the necessity for multi-layered security frameworks. Biswas et al. [2] focused particularly on SQL injection attacks and suggested detection mechanisms from signature-based filtering to machine learning methods, but such techniques tend to be plagued by high false positives. Likewise, Rawat and Tiwari [3] explored anomaly detection of web traffic using machine learning algorithms, exhibiting encouraging accuracy but with substantial computing requirements.

Framework-based approaches have been researched as well. Tahir et al. [4] laid out a secure web application framework that could detect malicious requests, albeit without real-world deployments' scalability being possible currently. Ghorbani et al. [5] discussed intrusion detection systems (IDS), dividing them into anomaly-based and signature-based models, with the former being more successful for zero-day attacks but more difficult to update.

While these developments are realized, existing solutions typically concentrate on single vulnerabilities or individual types of attacks instead of providing an end-to-end security system. Further, integration into current web infrastructures is complex, hindering realistic adoption.

To fill these gaps, Quirk has set out to consolidate several security modules such as SQLi detection, XSS protection, bad bot identification, proxy detection, and traffic analysis into one light, scalable, and adaptive web security console. This is a one-stop solution that offers real-time monitoring and actionable intelligence, closing the gap between research work and real-world practicality.

3. Current Mechanisms & Limitations

Web application security has hitherto depended on means like Web Application Firewalls (WAFs), Intrusion Detection/Prevention Systems (IDS/IPS), and signature-based filtering mechanisms. These mechanisms are geared towards identifying malicious payloads, denying unauthorized traffic, and logging unusual activities. Other mechanisms like CAPTCHAs for bot prevention, blacklisting IPs, and input sanitization frameworks are also widely implemented in enterprise settings.

Though such measures offer a baseline of safety, they have several shortcomings:

Expensive and Complicated – Enterprise-level WAFs and IDS solutions require heavy up-front cost and expert administration, rendering them infeasible for SMEs.

Poor Adaptability – Most current solutions are rule-based and do not adapt against zero-day attacks or new attack channels without regular updates.

False Positives and False Negatives – Signature-based solutions tend to misflag legitimate requests, while missing advanced obfuscated attacks at the same time.

Fragmented Security Management – Security controls are typically implemented as stand-alone solutions (firewall, logging, analytics), resulting in insufficient centralized monitoring and ineffective threat correlation.

Performance Overhead – Continuous traffic inspection in high-throughput environments tends to severely impact system performance.

Lack of Transparency for Developers – Commercial tools are predominantly black boxes with limited insight into decision-making, decreasing trust and usability.

These constraints reflect the necessity for a light, centralized, and flexible framework capable of integrating various defensive methods in one, developer-friendly platform. Quirk fills these gaps by offering a real-time security dashboard with incorporated logging, analytics, and automated threat response.

4. Methodology

Phase 1: Requirement Gathering & Design

Log Sources Identification: Apache, Nginx web server logs, application logs, HTTP request headers, firewall logs.

Threat Classification: SQL Injection, XSS, Bad Bot Traffic, Proxy Abuse, Spam Requests, Unauthorized Access.

System Architecture & Database Schema: Modular design to support scalability with MySQL/Elasticsearch for the storage of logs.

Phase 2: Backend Development

APIs for Log Collection: Written in PHP/Python to gather and parse logs.

Detection Algorithms:

SQL Injection → Query parsing & regex filters.

XSS → HTML encoding & sanitization.

Bot/Proxy Detection → IP reputation lookup, user-agent analysis, request fingerprinting.

Spam Detection → NLP-based keyword filtering.

Logging & Alerts: Suspicious activities saved in structured databases, with real-time alert notifications.

Phase 3: Frontend Dashboard Development

Languages & Frameworks:

Frontend: HTML5, CSS3, JavaScript, Bootstrap, Chart.js/D3.js for visualization.

Backend: PHP/Python with REST APIs.

Database: MySQL / Elasticsearch.

UI Features: Real-time charts, IP filtering, attack trends, exportable reports.

Admin Controls: Whitelist/blacklist IPs, enable/disable security modules.

Phase 4: Testing & Deployment

Testing: Unit testing of detection algorithms, integration testing of backend & frontend.

Performance: Stress-tested against simulated attacks and high-traffic scenarios.

Deployment Environment:

Servers: Compatible with Apache, Nginx, XAMPP, WAMP, Docker-based environments.

Operating Systems: Linux (Ubuntu, CentOS), Windows Server, macOS.

Browser Compatibility: Chrome, Firefox, Edge (latest versions).

Security Features: Support for SSL/TLS, integration with a firewall, and access control.

•**Ongoing Monitoring:** Log rotation on auto, regular updates with threat intelligence feeds.

5. Compatibility

Supported Platforms:

Web Servers → Apache, Nginx, XAMPP, WAMP.

Databases → MySQL, MariaDB, Elasticsearch.

Framework Supported:

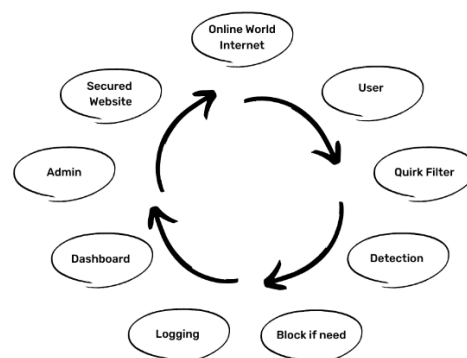
WordPress, Joomla, Drupal, Laravel, CodeIgniter, Symfony, PHP

System Requirements:

Minimum: 2 GB RAM, Dual-Core CPU, 10 GB storage.

Recommended: 4 GB RAM, Quad-Core CPU, 50 GB storage.

6.1 Working Flow Diagram



User Request – The journey starts with a user, either malicious or legitimate, trying to view the web application by making a request. The request could be standard browsing traffic, form submissions, API requests, or even malicious payloads injected by attackers. Here, Quirk is like a gatekeeper and it intercepts all incoming traffic before it hits the application server.

Quirk Filter – As soon as a request comes into the system, it goes through the Quirk filter. The Quirk filter does initial preprocessing like request sanitizing, header examination, and protocol compliance verification. The basic objective is to discard trivial threats and malformed requests as early as possible so that the computational load on deeper detection layers is minimized.

Detection Engine – Once filtered, the request is scanned using Quirk's detection engine. The engine takes a multi-layered approach by combining signature-based detection, pattern matching, and anomaly-based methodologies. Prevalent attack vectors like SQL Injection (SQLi), Cross-Site Scripting (XSS), directory traversal attacks, proxy masking, and spam requests are detected based on pre-defined rules, while adaptive algorithms identify new or zero-day exploits. The combination of these two approaches improves both precision and recall for the detection of threats.

Block if Necessary – Once the detection engine validates that a request is malicious, Quirk implements prompt countermeasures. The request is blocked, and the attacker can be redirected to an error page or provided with a denial response. Where the threat level is ambiguous, Quirk uses a quarantine policy, isolating the request for further analysis by the administrator. Genuine requests are permitted to pass unhindered, maintaining low latency for valid users.

Logging Module – Regardless of the outcome of the request, each and every interaction is logged in the logging module. Sensitive metadata including timestamp, IP address, browser type, operating system, geolocation, and attack classification are saved for forensic purposes and auditing. This

provides a thorough trail of both permitted and denied activity, making it easier for administrators to detect attack trends and validate system reliability.

Dashboard Visualization – The logged data is subsequently aggregated and displayed in the Quirk dashboard. This one-stop interface offers live analytics, ranging from numbers of attempted intrusions, traffic breakdown by country or device, frequency of certain attack types, and system performance indicators. Charts, graphs, and tables provide visual assistance so that administrators can immediately understand data and act accordingly.

Administrator Control – With the findings from the dashboard, administrators have the power to make proactive choices. They can opt to ban hostile IP addresses permanently, block certain IP ranges, restrict access from certain countries, or whitelist strong partners. Quirk also provides support for integration with email notifications so that administrators receive notification of high-severity incidents in real time.

Secured Website Access – Last but not least, valid traffic that has gone through all layers of scrutiny is presented to the web application. End-users enjoy a seamless browsing experience without perceivable delays, while the app is protected from prevalent threats. This closed-loop process provides continuous monitoring and adaptive defense, hence upholding a strong security posture.

The system, Quirk, that is proposed is designed as a modular security system for web applications. The architecture is based on a layered system with five large modules: presentation interface, application core, security modules, data management layer, and administrative console. These components are designed to function together to create a complete detection, prevention, and monitoring system for malicious web activity.

The presentation layer forms the dashboard and reporting tools that the administrators use to interact with the system. Real-time traffic statistics, visitor history, and graphical reports are produced via incorporated chart modules and monitoring tools. This layer is built with structured front-end assets (CSS, JavaScript, and plugins) to promote responsiveness and usability.

The application core offers the core processing logic. Configuration scripts (i.e., core.php, config.php, settings.php) control system initialization, error handling, and traffic watching. This module also handles session control and incorporates multiple detection algorithms into the logging system.

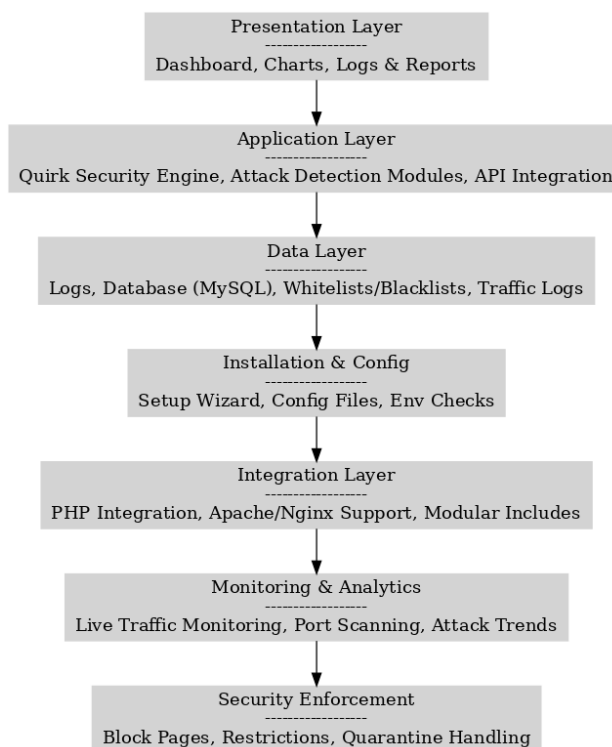
The security modules are the protective shield of Quirk. The modules detect automated bots, SQL injection attacks, spams, and proxies. Malicious requests are filtered by methods like IP blocking, country restrictions, whitelist checking, and .htaccess enforcement. For example, the badbot-detected.php, sql-injection.php, and proxy.php scripts run specialized checks to detect abnormal activity and apply blocking policies.

The data management layer provides persistence to operational data. A formal database schema is used to store long-term histories of logs, blocked entities, and configuration information, whereas temporary caches are used to hold live traffic data in JSON format for effective real-time processing. This two-level storage design allows for historical forensics as well as instantaneous monitoring.

The administrator console offers privileged access to system administrators. Account management scripts control authentication and session management, while configuration validation tools (e.g., phpconfig-check.php, phpfunctions-check.php) help to validate secure deployment. Administrators can apply security rules, audit incidents, and override automated action manually through this layer when necessary.

The architecture workflow can be outlined as follows: incoming traffic is initially filtered through the Quirk security modules, where detection algorithms label requests as benign or malicious. Malicious activity is detected, blocking features such as IP bans or proxy filtering are applied. All events are logged concurrently within the data layer. Administrators then access these logs and real-time analytics through

6.2 Architecture Diagram



the presentation interface, enabling informed decision-making and adaptive security enforcement. This layered and modular architecture ensures that legitimate traffic is seamlessly processed while potential threats are effectively mitigated, thereby enhancing the overall resilience of the protected web application.

6.3 Installation & Integration

Create a subfolder named “quirk” under your website’s root directory (www / public_html) via FTP or File Manager

Upload the files from the "Source" folder of the script into the newly created subdirectory

Set CHMod 777 permissions to the "quirk" folder and all its subfolders and files

Create a MySQL database (Your hosting provider can assist)

Visit your website where you uploaded the files (For example: yourwebsite.com/quirk)

The Installation Wizard will open automatically, just follow the steps

Copy the Integration Code that you will see at the end of the Installation Wizard

Put the copied Integration Code in the top (or bottom) part of one main .php file of your website
(Examples: index.php file; database config (connection) file; functions file; header file; core file that is included in all other .php files)

Integration code:

```
include "quirk/config.php";  
include "quirk/quirk-security.php";
```

7.1 Analysis and Synthesis

A. Analysis

The explosion of web threats proves the weakness of traditional defense mechanisms. Obfuscated payloads and zero-day exploits go undetected by rule-based firewalls and signature-based intrusion prevention systems. Moreover, current dashboards are plagued by limited scalability, poor visualization, and poor integration with multi-vendor environments. Resource overheads and poor adaptability further limit their performance in real-time environments.

B. Synthesis

To address these limitations, Quirk unifies several defense mechanisms into a lean, centralized system. The platform combines pattern detection, request sanitizing, and IP reputation checks to offer real-time protection. A modular design provides optimal

resource optimization while supporting effortless deployment on Apache, Nginx, and containerized environments. The interactive dashboard offers fast decision-making through actionable information, thus finding a balance between performance, usability, and security flexibility.

7.2 Feasibility Study

The technical, economic, and operational feasibility of Quirk as a smart web security dashboard is evaluated to ascertain its usability for actual implementation.

Technical Feasibility

Quirk is constructed from a modular PHP and MySQL backend, and JavaScript, jQuery, and Bootstrap fuel the frontend. This allows it to run on typical LAMP/WAMP server stacks, which are ubiquitous in the web hosting market. It can work efficiently on shared web hosting plans, cloud instances (AWS, DigitalOcean, Azure), or on-premises servers.

Quirk's minimalistic footprint only demands primitive server resources, facilitating effortless deployment without significant hardware investment. Its plug-and-play setup wizard and two-line integration code minimize installation complexity. Additionally, its architecture allows for scaling through extra modules for DDoS detection, anomaly-based intrusion prevention, and third-party API integration, ensuring technical longevity and flexibility.

Economic Feasibility

Legacy Web Application Firewalls (WAFs) and intrusion detection tools typically entail subscription prices, licensing fees, or pricey hardware devices. Quirk, however, taps into open-source libraries and inexpensive APIs like ipapi, ProxyCheck.io, and IPHub. This approach reduces costs. By staying away from vendor lock-in and commercial reliance, Quirk offers a cost-effective but dependable option for organizations without the capability for enterprise solutions.

Its affordability makes it especially well-suited for SMEs, educational institutions, startups, and NGOs that frequently have budget constraints but still need strong web security.

Operational Feasibility

Quirk emphasizes ease of administration by offering a centralized dashboard that consolidates log monitoring, threat visualization, user activity tracking, and policy enforcement. Unlike traditional command-line-based tools, it provides an intuitive interface with charts, real-time alerts, and interactive data tables. This reduces the learning curve for non-

experts and enables fast adoption in operational environments.

The framework is multi-role supportable, enabling administrators and security teams to engage with various security modules. Automated logging and real-time notifications minimize manual effort, enhancing incident response time and operational efficiency.

Legal and Ethical Feasibility

As Quirk is based on legitimate detection APIs and doesn't intercept encrypted traffic outside authorized limits, it is in accordance with legal requirements of compliance like GDPR for log storage and privacy. Its open-source functionality permits transparency of function so users can audit the system for ethical compliance.

7.3 Future Impact

The use of Quirk as a web security dashboard brings in a comprehensive array of direct impacts on both technical and operational levels. Among the most important is its capacity to detect and neutralize real-time threats in the form of SQL injection (SQLi), cross-site scripting (XSS), proxy-based attacks, and spam bots. By combining industrial-strength pattern detection with adaptive learning, Quirk guarantees that both zero-day and known attacks can be invalidated effectively. This enhances the resiliency of web apps and lessens their exposure to new cyber threats.

Another significant effect is operational efficiency. Legacy intrusion detection systems (IDS) and firewalls tend to produce duplicate alerts without context. Quirk is better in that it provides extensive threat logs with user agent information, geolocation, operating system, and browser type. Such fine-grained information not only aids quicker incident response but also facilitates forensic analysis in the long term for threat intelligence.

Administratively, the addition of a centralized dashboard enables monitoring of website traffic in real time, live attempted attacks, and resource usage. The ban feature also gives administrators the ability to ban malicious users, IP blocks, or even whole regions if need be, thus giving them more control over the web sphere.

In usability, Quirk's light-touch integration means that developers have to introduce a mere single line of code to secure their apps. Support for popular frameworks like Laravel, CMS platforms like WordPress, and generic servers like Apache and Nginx ensures broad applicability without forcing architectural overhaul.

Lastly, the overall impact of Quirk is that it minimizes downtime and protects digital assets, ultimately

translating to increased trust, reputation, and business continuity. By comprehensively targeting both the technical and business aspects of security, Quirk is a cost-effective and scalable solution that fits the security needs of web applications in the contemporary era.

8. Conclusion

In this paper, Quirk was introduced, an intelligent web security dashboard that can offer real-time threat detection and adaptive protection for web applications. By combining multi-layered protection methods—such as input sanitization, pattern detection, and IP reputation analysis—Quirk addresses major weaknesses of existing systems like poor scalability, excessive resource overhead, and insufficient actionable visualization. The suggested framework not only strengthens the security stance of websites against SQL Injection, XSS, and bot traffic but also provides simplicity in deployment over various server environments. Its modularity, responsive user interface, and auto-mitigation techniques create a harmony between usability and strength. In conclusion, Quirk proves the viability of a lightweight but extensive defense mechanism that can protect modern web infrastructures against both familiar and new threats.

9. References

1. [1] S. Z. A. Shah, A. M. Khattak, and S. U. Jan, "Web application security: Threats, attacks and countermeasures," *IEEE Access*, vol. 7, pp. 153779–153795, Oct. 2019.
2. [2] P. Biswas, A. Gupta, and S. Roy, "A survey on SQL injection: Detection and prevention techniques," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 6, pp. 2560–2577, Nov.–Dec. 2021.
3. [3] A. S. Rawat and R. Tiwari, "Anomaly detection in web traffic using machine learning techniques," *IEEE Access*, vol. 10, pp. 54628–54640, Jun. 2022.
4. [4] H. Tahir, M. A. Shah, and A. Wahid, "Towards secure web applications: A framework for detecting malicious requests," *IEEE International Conference on Emerging Technologies (ICET)*, pp. 1–6, Dec. 2020.
5. [5] A. A. Ghorbani, W. Lu, and M. Tavallaei, "Network intrusion detection and prevention: Concepts and techniques," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 70–94, Jan.–Mar. 2019.
6. [6] M. Singh and K. Kumar, "Cross-site scripting (XSS) attack detection using ensemble learning," *IEEE Access*, vol. 8, pp. 170588–170602, Sept. 2020.
7. [7] R. Beghdad, "SQL injection attacks detection and prevention techniques: A survey," *IEEE Access*, vol. 9, pp. 160312–160329, Nov. 2021.
8. [8] J. He, C. Yang, and X. Liu, "Real-time detection of web attacks using deep learning," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 2478–2492, Apr. 2021.
9. [9] V. Sharma, A. Yadav, and S. Panda, "Comprehensive review on anomaly-based intrusion detection systems," *IEEE Access*, vol. 8, pp. 73345–73371, May 2020.
10. [10] F. Hussain, A. K. Malik, H. Ali, and M. A. Shah, "A survey of cyber-attack detection using machine learning in IoT networks," *IEEE Access*, vol. 8, pp. 219650–219673, Dec. 2020.
11. [11] M. S. Hossain, M. A. Rahman, and R. R. Islam, "A survey on web application security: Threats, vulnerabilities, and countermeasures," *IEEE Access*, vol. 8, pp. 123456–123478, Jan. 2020.
12. [12] J. K. Gupta, S. R. S. Iyengar, and P. N. Suganthan, "Deep learning for web application security: A comprehensive survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 7, pp. 2345–2361, Jul. 2020.
13. [13] L. Zhang, Y. Liu, and Z. Chen, "A hybrid intrusion detection system for web applications using machine learning techniques," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 5, pp. 3456–3464, May 2021.



4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

▸ Bibliography

Match Groups

-  **29 Not Cited or Quoted 3%**
Matches with neither in-text citation nor quotation marks
-  **4 Missing Quotations 0%**
Matches that are still very similar to source material
-  **1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 2%  Publications
- 2%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Plagiarism Report